



Escuela
Politécnica
Superior

Extracción y Estructuración de Contenido Lingüístico en PDFs para la Preservación de Lenguas Mayas



Máster en Inteligencia Artificial

Trabajo Fin de Máster

Autor:

Elías Alfonso Carrasco Guerrero

Tutor:

Juan Antonio Pérez Ortiz

Setiembre 2025



Universitat d'Alacant
Universidad de Alicante

Digitalización, estructuración y preservación lingüística de gramáticas mayas minorizadas mediante inteligencia artificial multimodal

Automatización de la preservación de lenguas minorizadas: un enfoque basado en modelos de lenguaje y marcado XML

Autor

Elías Alfonso Carrasco Guerrero

Directores

Juan Antonio Pérez Ortiz

Departamento de Lenguajes y Sistemas Informáticos



MÁSTER EN INTELIGENCIA ARTIFICIAL



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

Alicante, 3 de junio de 2026

Resumen

La aplicación desarrollada para este Trabajo de Fin de Máster consiste en un software modular programado en Python que se ejecuta desde la terminal de comandos. Su función principal es automatizar la extracción de texto y la comprensión visual de páginas en formato PDF pertenecientes a gramáticas de lenguas mayas. El sistema inyecta ejemplos estructurados de entrada y salida (few-shot) junto a un prompt especializado directamente en la memoria de la API de Gemini, logrando que el modelo de Inteligencia Artificial entienda la disposición tipográfica original y devuelva la información totalmente organizada en un archivo XML con etiquetas semánticas personalizadas (como `¡morpheme.analysis!` o `¡syntactic.analysis!`). Posteriormente, el pipeline procesa este archivo XML para generar de forma automática un documento en LaTeX que se compila para reconstruir la obra original en un nuevo archivo PDF con un acabado editorial profesional. El objetivo último es transformar "papel digital. estático en un conjunto de datos estructurados de alta precisión (Gold Standard) que sirva como combustible indispensable para que, en el futuro, otras Inteligencias Artificiales puedan entrenarse y aprender idiomas minorizados de bajos recursos computacionales. Este Trabajo de Fin de Máster aborda la problemática de la digitalización y estructuración de gramáticas normativas de lenguas mayas minorizadas (como Kaqchikel, K'iche' y Mam), cuyos documentos originales presentan estructuras tipográficas complejas que dificultan el procesamiento mediante técnicas de OCR convencionales. Se propone y desarrolla un sistema automatizado basado en Modelos de Lenguaje de Gran Tamaño (LLMs) a través de la API de Gemini, integrado en un pipeline modular desarrollado en Python. Con este flujo se consigue mitigar la pérdida de información semántica y espacial inherente a los métodos planos tradicionales, salvaguardando recursos científicos de alto valor como las glosas interlineales. La metodología se fundamenta en la transformación de documentos PDF en datos estructurados bajo el estándar XML, garantizando la integridad jerárquica mediante la definición de un Documento de Definición de Tipo (DTD) y el desarrollo de módulos específicos de corrección estructural (`xml.corrector.py`). El núcleo de la investigación se centra en un estudio empírico sobre el parámetro de temperatura del modelo, evaluando su impacto en la precisión y estabilidad del sistema bajo una estricta economía de tokens impuesta por las limitaciones de cuota del entorno operativo. Los resultados experimentales demuestran la existencia de un "punto dulce" (sweet spot) en temperaturas de 0.1 y 0.25, donde se alcanzan precisiones superiores al 90

Palabras clave: Inteligencia Artificial, LLM, Gemini API, Lenguas Mayas, Digitalización, XML, Procesamiento de Lenguaje Natural, OCR Estructurado.

Motivación y justificación

La preservación de la diversidad lingüística constituye uno de los desafíos más críticos en la era de la información. Las lenguas minorizadas, en particular aquellas pertenecientes a la familia maya como el Kaqchikel, el K'iche, el Mam y el Kanjobal, representan un patrimonio cultural de valor incalculable. Sin embargo, su supervivencia no sólo depende de su uso oral, sino también de la disponibilidad y accesibilidad de recursos académicos y normativos en formatos digitales modernos. En este contexto, la Academia de Lenguas Mayas de Guatemala (ALMG) desempeña un papel crucial como entidad rectora en la promoción y estandarización de estos idiomas. A través de su repositorio institucional de documentos (disponible en <https://almg.org.gt/documentos/>), la ALMG ha realizado un esfuerzo significativo por poner a disposición del público gramáticas normativas y textos de referencia, tal y como se muestra en la figura 1. No obstante, gran parte de este conocimiento se encuentra consignado en archivos PDF que, si bien facilitan la difusión, actúan esencialmente como "papel digital": una representación estática de la información que carece de estructura semántica.

Por ejemplo, ya en marzo de 2025, tuvieron una actualización de la web en la que eliminaron el 70Esta limitación impide que los datos lingüísticos (ejemplos gramaticales, reglas sintácticas, morfología) puedan ser procesados automáticamente por herramientas computacionales, buscadores especializados o sistemas de aprendizaje profundo. La motivación técnica de este trabajo radica, por tanto, en la superación de las barreras impuestas por el reconocimiento óptico de caracteres (OCR) convencional sobre este corpus específico de la ALMG, para generar corpus con morfologías clasificadas para entrenamiento de modelos. Las gramáticas lingüísticas de estas lenguas presentan una complejidad tipográfica elevada, que incluye tablas anidadas, glosas interlineales y estructuras jerárquicas que los sistemas tradicionales suelen interpretar de manera errónea. Ante este escenario, surge la oportunidad de emplear Modelos de Lenguaje de Gran Tamaño (LLMs), para comprender la arquitectura lógica de estos documentos institucionales y transformarla en datos estructurados bajo el estándar XML. Este proyecto busca proporcionar una solución eficiente que reduzca la carga de trabajo manual en la digitalización del patrimonio de la ALMG, garantizando que la información sea interoperable, consultable y apta para la generación de documentación científica profesional mediante LaTeX.

Documentos Publicados



Según la Ley de la Academia de las Lenguas Mayas de Guatemala, establece que dentro de sus funciones está traducir y publicar, previo cumplimiento de las leyes de la materia, códigos, leyes, reglamentos y otros textos legales o de cualquier naturaleza que se juzgue necesario a los idiomas Mayas.

Además de crear, implementar e incentivar programas de publicaciones bilingües y monolingües, para promover el conocimiento y uso de los idiomas mayas para fortalecer los valores culturales.

Para lo anterior la Academia ha realizado varias publicaciones en diferentes idiomas Mayas, y algunas de ellas se presentan a continuación:

Para mayor información pueden ingresar a la página de [contacto](#)

Comunidad Lingüística	Tipo de Documento	Nombre del Documento (Idioma Maya)	Nombre del Documento (Idioma Español)	Descarga
Achi	Texto	jik'ob'al Ub'ee Tz'ib' Maya'ch'a'teem Achi	Gramática _Normativa Idioma Maya Achi	
Akateka	Texto	Ik'ti'ey Skuyb'anil	Fábulas	
Akateka	Texto	Ik'ti'ey Skuyb'anil	Fábulas (2024)	
Akateka	Texto	xolilal Yelalil Ya'ley Ti'e Akateko	Gramática Descriptiva Akateka	
Akateka	Texto	Tz'ib' b'il q'ichmam Akateka	Literatura Maya Akateka	
Akateka	Texto	Sb'i K'ultajb'al yet Q'ichmam Akateko	Toponimias Maya Akateko	

Figura 1.: Web de la ALMG

Agradecimientos

*A mi familia por estar ahí en todo momento.
A mi pareja por ayudarme en todo lo que puede y más.
A mi madre por dedicar su vida a sus hijos.
A mi padre por animarme con mi carrera en la ciencia.
A mis compañeros del Santander por guiarme y enseñarme cómo desarrollar software.
Sin ellos no habría podido hacer este trabajo.*

Pues hemos nacido para colaborar, al igual que los pies, las manos, los párpados, las hileras de dientes, superiores e inferiores. Obrar, pues, como adversarios los unos de los otros es contrario a la naturaleza.

Marco Aurelio (Meditaciones)

Solo los que se quemaron con el fuego aprendieron a usarlo.

Andrés Pedreño Muñoz
Cofundador de Torre Juana OST

Índice general

Resumen	v
1. Introducción	1
1.1. Definición del problema	1
1.2. Alcance y limitaciones	3
1.2.1. Restricciones de propiedad intelectual y publicación de datos . . .	3
2. Estado de la cuestión	7
2.1. Modelos de lenguaje de gran tamaño (LLMs) aplicados a OCR	7
2.2. APIs de LLMs	8
2.2.1. Ecosistema actual	8
2.2.2. Análisis de la Volatilidad y Restricciones de Acceso	8
2.2.3. Configuración de Modelos: Parámetro de Temperatura	9
2.3. Transfer Learning: Sinergias Lingüísticas y Extrapolación Global	10
2.3.1. Inferencia multilingüe y pesos compartidos	10
2.3.2. El valor de los datos estructurados (XML/LaTeX)	10
2.3.3. Democratización del conocimiento lingüístico	11
2.4. Impacto y costes de la pérdida de documentos	11
2.4.1. Riesgos en el ciclo de vida documental	11
2.4.2. Consecuencias operativas y regulatorias	12
2.5. Otras iniciativas	12
2.6. Tecnologías web	13
2.6.1. Metodos HTTP	14
2.6.2. API	14
2.6.3. REST	14
2.7. Herramientas para el desarrollo	14
2.7.1. Angular	14
2.7.2. React	16
2.7.3. Spring	17
2.7.4. Spring Boot	17
2.7.5. JPA	18
2.7.6. Maven	19
2.7.7. Thymeleaf	19
2.8. Ciberseguridad	20
2.8.1. Cifrado simétrico	20
2.8.2. Cifrado asimétrico	20
2.8.3. Cifrado de contraseñas	21

2.8.4. Capas de Seguridad en el login	21
2.9. Diseño software	22
2.9.1. Patrones de diseño software	22
2.9.2. Diagramas UML	23
A. Anexo I. Esquema de base de datos	31
B. Anexo II. Código de la aplicación SIDManager	33
Lista de acrónimos	58

Índice de figuras

1.	Web de la ALMG	VIII
1.1.	Comparación OCR vs. Kaqchikel	2
2.1.	Ejemplo de componentes de Angular con código Typescript.	15
2.2.	Ejemplo de DOM virtual de React hecho por Angular Minds (2024) . . .	16
2.3.	Capas de Spring Boot hecho por Fadatare (2025)	18
A.1.	Esquema de la base de datos	31

Índice de tablas

1.1. Comparativa de disponibilidad de modelos 2025-2026	4
2.1. Comparativa de proveedores de modelos de IA (Junio 2026)	9

Índice de Listados

B.1. Kaqchikel XML generado por el LLM a partir de la imagen de la página original	33
B.2. Función getNotReceivedDocuments del controlador	36
B.3. Función getNotReceivedDocuments del servicio	37
B.4. Función listDocuments del controlador	37
B.5. Función getFilteredDocuments del servicio	37
B.6. Función findFilteredDocuments del repositorio	38
B.7. Función checkDates del servicio	38
B.8. Función getCreateClientUserView del controlador	39
B.9. Función createClientUser del controlador	39
B.10.Función saveClientUser del controlador	40
B.11.Función getConfirmation del controlador	40
B.12.Función confirmUser del servicio	41
B.13.Función getLoginView del controlador	41
B.14.Función getRequestUpdatePasswordView del controlador	42
B.15.Función requestUpdatePassword del controlador	42
B.16.Función sendUpdatePasswordUrl del servicio	42
B.17.Función getConfirmationUrl del servicio	43
B.18.Función getUpdatePasswordView del controlador	43
B.19.Función getUpdatePasswordView del servicio	44
B.20.Función updatePassword del controlador	44
B.21.Función updatePassword del servicio	45
B.22.Clase DocumentModel	46
B.23.Clase SecurityConfig	47
B.24.Clase CustomUserDetailsService	49
B.25.Clase CustomUserDetailsService modificado	49
B.26.Clase CustomAuthenticationFailureHandler	50
B.27.Función encryptMessage de Encryptor	50
B.28.Función decryptMessage de Encryptor	51
B.29.La función getDataSource de DataSourceConfig	51
B.30.El contenido html para mostrar la tabla de documentos	52
B.31.El contenido html para conectar los links de Bootstrap 5	52
B.32.El contenido html para enviar un mensaje al front-end con Thymeleaf	53
B.33.La clase Mail	53
B.34.La clase ResponseEnum	57
B.35.La clase LenguajeConfig	57

1. Introducción

1.1. Definición del problema

La digitalización de obras lingüísticas complejas se enfrenta a un obstáculo fundamental: la dicotomía entre la representación visual y la estructura lógica. El problema central se desglosa en los siguientes puntos críticos:

1. Naturaleza heterogénea de las fuentes (PDFs no procesados)

A diferencia de los documentos digitales nativos, el corpus de trabajo presenta una gran disparidad en su calidad técnica. Se ha identificado que un volumen significativo de los archivos PDF no son documentos con texto seleccionable y fondo limpio, sino fotografías directas de las páginas del libro físico. Estas fuentes carecen de cualquier pre-procesamiento de imagen o capa de texto subyacente, lo que implica: Ruido visual: Presencia de sombras, texturas del papel y falta de contraste entre la tipografía y el fondo. Inexistencia de metadatos de texto: La imposibilidad de extraer caracteres mediante scripts sencillos de lectura de PDF, obligando al sistema a realizar una interpretación visual completa de la imagen.

2. Insuficiencia del OCR convencional

Los sistemas de OCR tradicionales están diseñados para la extracción de texto lineal en documentos con una disposición de página sencilla como la parte izquierda de la figura 1.1. Sin embargo, la amalgama de una baja calidad de imagen junto con la arquitectura tipográfica y análisis sintácticos y morfológicos de las gramáticas como en la parte derecha de la figura 1.1, genera fallos sistemáticos en:

- Sistemas de numeración jerárquica:

Divisiones en secciones, tablas y disposiciones en vertical que suelen confundirse con el texto horizontal.

- Glosas y ejemplos multilingües:

La alternancia entre el castellano y la lengua maya, a menudo en bloques adyacentes, que el OCR estándar tiende a mezclar al no reconocer los límites visuales de las columnas o tablas.

- Estructuras morfológicas complejas:

Paradigmas verbales que, al ser extraídos de una "foto como texto plano, se convierten en caracteres inconexos.

puede generar alucinaciones incluso empleando la metodología esquema de respuesta (response schema).

1.2. Alcance y limitaciones

El desarrollo de este Trabajo de Fin de Máster está condicionado por factores de carácter ético, legal y técnico que delimitan el marco de actuación. A continuación, se detallan las restricciones que han definido el alcance final de la investigación.

1.2.1. Restricciones de propiedad intelectual y publicación de datos

El corpus bibliográfico utilizado en este proyecto, consistente en gramáticas normativas de lenguas mayas, es propiedad intelectual de la Academia de Lenguas Mayas de Guatemala (ALMG). Como resultado de esta titularidad, se han establecido las siguientes limitaciones en cuanto a la difusión de datos:

- Prohibición de redistribución: No se permite la subida de los archivos PDF originales a repositorios públicos como GitHub. El acceso a las fuentes debe realizarse exclusivamente a través del portal institucional de la ALMG.
- Publicación de resultados parciales: El alcance de la publicación de resultados se limita a la estructura XML generada y a los fragmentos utilizados en el conjunto de pruebas (test set), preservando la integridad de la obra original.
- Lógica de replicabilidad: Para solventar esta limitación, el proyecto se enfoca en la liberación del código fuente (scripts de procesamiento), permitiendo que otros investigadores con acceso legítimo a los documentos puedan ejecutar el pipeline de digitalización de forma independiente.

Cabe destacar que el 70 % de los documentos mayas publicados en 2025 ya no están disponibles porque se han eliminado del catálogo de la web de la ALMG sin previo aviso. Esto provocó una gran dificultad en el trabajo ya que se tuvo que proseguir con los pocos documentos que se habían guardado en local antes de la actualización del catálogo.

Acotación de la carga de trabajo

De acuerdo con las directrices marcadas por la tutoría académica y con el fin de ajustar el proyecto a una carga de trabajo estrictamente acotada de 250 horas, se ha definido un alcance modular y selectivo:

- Selección de test set: En lugar de proceder a la digitalización de los libros completos, lo cual excedería el tiempo asignado en el cronograma, se ha optado por un análisis empírico profundo sobre una muestra representativa. Se han seleccionado un rango de 2 a 3 páginas de alta complejidad por cada idioma (Kaqchikel, K'iche, Mam y Qanjobal) para conformar el conjunto de validación final (Ground Truth).

- **Priorización técnica:** Se ha priorizado el perfeccionamiento de los módulos de validación estructural (`xml_corrector.py`) y la traducción automatizada a LaTeX sobre el procesamiento masivo de volumen, garantizando la robustez de la arquitectura antes de su escalabilidad futura.

Limitaciones técnicas de la API de Gemini

La dependencia de servicios de inteligencia artificial en la nube ha supuesto uno de los mayores desafíos logísticos del proyecto, debido a cambios imprevistos en las políticas de uso de Google Cloud y AI Studio, así como a la obsolescencia programada de sus arquitecturas:

- **Restricción drástica de cuotas de uso:** Durante el transcurso del desarrollo, el sistema sufrió una reducción crítica en los límites de peticiones permitidas. Se pasó de un margen operativo de 1.500 usos diarios (Requests Per Day - RPD) a un límite restrictivo de apenas 20 peticiones al día para los modelos en el nivel gratuito tal y como se ve en la tabla 1.1. Esta limitación del 98.6 % en la capacidad de cómputo obligó a reestructurar el flujo de trabajo, priorizando ejecuciones de prueba extremadamente precisas y limitando la posibilidad de realizar procesamientos masivos de páginas en una sola jornada.

Tabla 1.1.: Comparativa de disponibilidad de modelos 2025-2026

Modelos	Disponibilidad 2025 RPD	Disponibilidad 2026 RPD
gemini 1.5 flash	500	Deprecated
gemini 1.5 flash-lite	1500	Deprecated
gemini 2.0 flash	200	Deprecated
gemini 2.0 flash-lite	500	Deprecated
gemini 2.5 flash	20	20
gemini 2.5 flash-lite	20	20
gemma 3 27b	15.000	Deprecated
gemma 4 27b	No se había lanzado	Solo texto (al quitarle PDF)
gemini-3-flash-preview	No se había lanzado	20
gemini-3.1-flash-lite	No se había lanzado	500
gemini-3.5-flash	No se había lanzado	20

- **Reducción en la disponibilidad y ciclo de vida de los modelos:** Inicialmente, el entorno permitía el acceso concurrente a una variedad de entre 4 y 5 modelos de lenguaje distintos (incluyendo versiones experimentales y estables). No obstante, esta disponibilidad se vio mermada a solo 1 o 2 modelos activos por ventana de desarrollo debido a la depreciación masiva de arquitecturas estables. Específicamente, hitos de la plataforma como el del 29 de septiembre de 2025 supusieron la eliminación silenciosa de modelos de referencia como `gemini-1.5-pro` y `gemini-1.5-flash`, restringiendo la capacidad de realizar comparativas directas de rendimiento intramodelo de forma simultánea.

- Impacto en la metodología y presupuesto de tokens: Estas limitaciones técnicas externas impusieron una estricta economía de tokens y de tiempo. Se ha calculado matemáticamente el coste por llamada del pipeline mediante la ecuación de consumo estructural de tokens 1.3, considerando el prompt base y las capas de información visual y estructural que se envían a la API en cada iteración.

$$\text{Tokens por llamada} = 5,000 \text{ (prompt)} \quad (1.1)$$

$$+ [2,000 \text{ (imagen PDF)} + 2,000 \text{ (esquema XML)}] \times 2 \quad (1.2)$$

$$= 13,000 \text{ tokens} \quad (1.3)$$

Considerando las restricciones de la API para cuentas de desarrollo sin facturación asociadas a un techo de 250.000 tokens por minuto (Tokens Per Minute - TPM) y un límite de 10 solicitudes por minuto (RPM), la ventana de transmisión quedó restringida físicamente a un máximo de 19 llamadas simultáneas por minuto, forzando la inclusión de retrasos controlados en el script orquestador de 10 segundos en los reintentos. Sin embargo, tal y como se ha comentado anteriormente, la limitación más restrictiva fue la de peticiones por día (RPD - Request Per Day), ya que es más probable alcanzar 20 llamadas en un día que 19 llamadas en un minuto.

- Otra problemática de la API de gemini es que en momentos de alta demanda, esta no te devuelve resultados, te da error de servidor sobrecargado, lo que ha obligado a que en el proyecto se controle este flujo implementando un reintento a las 15 segundos.

Ante la inviabilidad operativa de las cuotas comerciales y los intentos fallidos de implementar arquitecturas locales basadas en la familia Gemma —debido a que gemma-3 fue completamente retirado por el proveedor y gemma-4 arrojó errores de servidor 500 al procesar de forma nativa la capa visual de los PDFs—, se optó estratégicamente por reorientar el pipeline hacia un modelo alternativo con límites específicos: gemini-3.1-flash-lite. Este cambio permitió eludir el bloqueo de cuota diaria (al ofrecer un techo de 500 peticiones), asumiendo una penalización marginal del 10% en la tasa de error de caracteres (CER) frente a los modelos de mayor escala, lo que obligó a robustecer los algoritmos heurísticos de post-procesamiento en el core del sistema.

2. Estado de la cuestión

2.1. Modelos de lenguaje de gran tamaño (LLMs) aplicados a OCR

Tradicionalmente, el Reconocimiento Óptico de Caracteres (OCR) se ha abordado mediante técnicas de visión artificial centradas en la detección de patrones de píxeles y la segmentación de glifos tal y como sostiene Singh et al. (2015). Herramientas clásicas como Tesseract o motores basados en redes neuronales convolucionales (CNN) han demostrado una alta eficacia en textos lineales y documentos digitales nativos. Sin embargo, su rendimiento decrece significativamente ante documentos con estructuras tipográficas complejas, ruido visual o disposiciones no lineales de la información. En los últimos años, la irrupción de los Modelos de Lenguaje de Gran Tamaño (LLMs) ha provocado un cambio de paradigma, evolucionando hacia lo que se conoce como OCR Estructurado o Document AI. A diferencia del OCR convencional, que se limita a transcribir caracteres, los LLMs con capacidades multimodales (Visual-Language Models o VLMs) procesan el documento como un todo semántico (Shen et al., 2025). Esta metodología se fundamenta en los siguientes pilares:

- Comprensión contextual y multimodalidad: Los modelos de última generación, como la familia Gemini empleada en este trabajo, integran codificadores visuales y de lenguaje. Esto permite al sistema no solo "ver" los caracteres, sino comprender la relación espacial entre ellos (Ding et al., 2026).

En el contexto de las gramáticas mayas, esta capacidad es crítica para diferenciar una glosa interlineal de un cuerpo de texto estándar, basándose en la disposición física en la página.

- Capacidad de inferencia y reconstrucción: A diferencia de los sistemas tradicionales que fallan ante caracteres ambiguos o manchas en el papel (comunes en las fotos de libros de la ALMG), los LLMs utilizan su conocimiento previo del lenguaje para inferir palabras probables, actuando simultáneamente como motor de extracción y corrector ortográfico contextual (Shen et al., 2025).
- Generación de salida estructurada: Uno de los mayores avances aplicados a este proyecto es la capacidad de estos modelos para seguir instrucciones complejas de formato (prompting). Mientras que un OCR tradicional devuelve texto plano (.txt), los LLMs pueden ser instruidos para encapsular la información directamente en lenguajes de marcado como XML. Esto elimina la necesidad de capas interme-

días de análisis de layout (Layout Analysis), permitiendo que el modelo genere la jerarquía de capítulos, secciones y subsecciones de forma nativa.

- Resiliencia ante fuentes heterogéneas: Se ha observado que los LLMs presentan una robustez superior al procesar documentos que carecen de metadatos de texto, como es el caso de las capturas fotográficas de libros antiguos. Su entrenamiento masivo en conjuntos de datos diversos les permite normalizar variaciones en la iluminación, curvatura de página y texturas del papel que suelen ser insalvables para algoritmos de binarización clásicos.

No obstante, la aplicación de LLMs al OCR introduce desafíos específicos, principalmente su naturaleza estocástica. Al ser modelos probabilísticos, existe el riesgo de .^alucinaciones.^{es}tructurales (etiquetas XML inexistentes o mal cerradas). Esta limitación justifica la implementación de módulos de validación externa y corrección estructural, tales como el script `xml_corrector.py` desarrollado en este TFM, para garantizar que la potencia de comprensión del modelo se ajuste estrictamente a la gramática formal (DTD) requerida.

2.2. APIs de LLMs

2.2.1. Ecosistema actual

En el ecosistema tecnológico de 2026, el acceso a Modelos de Lenguaje de Gran Tamaño se realiza mayoritariamente a través de interfaces de programación de aplicaciones (APIs) bajo un modelo de computación en la nube (Serverless AI). Este paradigma permite a los desarrolladores integrar capacidades cognitivas avanzadas sin la necesidad de gestionar infraestructura de hardware costosa. Sin embargo, el mercado se caracteriza por una alta volatilidad en los precios y, especialmente, en las políticas de cuotas de uso gratuito para investigación académica. En la tabla 2.2.1, se presenta una comparativa de las principales APIs disponibles, considerando las versiones de los modelos que lideran el sector en el presente año usando como fuente la web Cloudzero - LLM API Comparison In 2026.

2.2.2. Análisis de la Volatilidad y Restricciones de Acceso

Uno de los hallazgos más relevantes durante el desarrollo de este TFM ha sido la inestabilidad de los servicios API como infraestructura para la digitalización lingüística. A diferencia de años anteriores, donde las cuotas para desarrolladores eran amplias, en 2026 se ha detectado una política de .^ajuste agresivo” por parte de los grandes proveedores (específicamente Google Cloud/AI Studio). Se ha documentado, mediante la bitácora de desarrollo de este trabajo, que las condiciones de acceso para el modelo Gemini sufrieron una degradación crítica sin previo aviso:

- Colapso de cuota: El margen operativo se redujo de 1.500 peticiones diarias a solo 20, lo que representa una caída del 98.6

Proveedor / Modelo	Plataforma	Req/día	Req/min	Tokens/min	Ventana	Max. Salida	Notas clave
gemini-2.5-flash	Google API	20	15	1M	1M	8K	Tier gratuito permanente.
gemini-3.1-flash-lite	Google API	1500	15	250K	1M	8K	Límites restrictivos en free tier.
gemini-2.0-flash	Google API	20	15	1M	1M	8K	Multimodal. Opción legacy.
llama-3.3-70b-versatile	GroqCloud	1,000	30	6,000	128K	32K	Velocidad 300-400 TPS.
llama-3.1-8b-instant	GroqCloud	14,400	30	30,000	128K	8K	El más permisivo de Groq.
llama-4-scout-17b	GroqCloud	1,000	30	30,000	128K	16K	Arquitectura nueva.
llama-3.3-70b	Cerebras Cloud	1M tok/día	30	60k-100k	8K	8K	2,600 TPS.
qwen3-32b	Cerebras Cloud	1M tok/día	30	60,000	64K	32K	Pool compartido.
mistral-small-3.1	La Plateforme	1B tok/mes	500	500,000	128K	8K	1B tokens/mes.
DeepSeek v3 / r1	DeepSeek Platf.	Sin límite	Dinám.	Dinám.	128K	8K	5M tokens registro.
gpt-4o / mini	OpenAI API	—	—	—	128K	16K	Requiere depósito.
claude-sonnet/haiku	Anthropic API	—	—	—	200K	8K	No tiene API free.

Tabla 2.1.: Comparativa de proveedores de modelos de IA (Junio 2026)

- Restricción de modelos: La oferta de arquitecturas disponibles para pruebas concurrentes pasó de 8 modelos activos a únicamente 5, limitando la capacidad de realizar estudios comparativos de rendimiento intramodelo.

Esta situación subraya la importancia del objetivo específico 6 de este trabajo: la evaluación de la portabilidad hacia modelos locales. La dependencia de APIs comerciales, aunque ofrece una potencia de inferencia superior y capacidades multimodales nativas para procesar fotos de libros, introduce un riesgo de .“apagón técnico” que puede comprometer cronogramas de investigación académica estricta. En conclusión, mientras que APIs como DeepSeek ofrecen el mejor ratio coste-beneficio en términos de tokens, Google sigue liderando en ventana de contexto, a pesar de la fragilidad de sus cuotas para el estamento de usuarios no corporativos.

2.2.3. Configuración de Modelos: Parámetro de Temperatura

Para optimizar el rendimiento de un LLM en tareas de digitalización estructurada, no solo es determinante la elección de la API, sino también el ajuste de sus hiperparámetros de generación y la selección de la arquitectura adecuada para la carga de trabajo.

La temperatura es un hiper parámetro que regula el grado de aleatoriedad o “creatividad” del modelo durante la fase de muestreo de tokens. Normalmente se define con un número en el rango del 0 al 1 (siendo 0 el mínimo y 1 el máximo grado de temperatura), aunque en gemini se regula de 0 a 2. Técnicamente, actúa sobre la función softmax de la capa de salida del modelo, modificando la distribución de probabilidad de las palabras

candidatas:

- Temperaturas bajas (T aprox 0.0 a 0.3): El modelo se comporta de forma cuasi-determinista, seleccionando siempre los tokens con mayor probabilidad estadística. En el contexto de este TFM, este rango es esencial para garantizar que la estructura XML sea coherente y que la transcripción de las gramáticas mayas sea fiel al original, minimizando la "alucinación" de etiquetas.
- Temperaturas altas ($T \geq 0.7$): El modelo aplanar la curva de probabilidad, otorgando oportunidades a tokens menos frecuentes. Aunque esto favorece la creatividad en redacción literaria, en la digitalización de documentos técnicos suele derivar en el colapso estructural del XML (etiquetas mal cerradas o duplicadas) y en la invención de términos lingüísticos inexistentes.

2.3. Transfer Learning: Sinergias Lingüísticas y Extrapolación Global

El aprendizaje por transferencia (Transfer Learning) representa uno de los pilares más avanzados de la Inteligencia Artificial moderna. Se define como la capacidad de un modelo de computación neuronal para aplicar el conocimiento adquirido en una tarea previa (fuente) a una tarea nueva pero relacionada (objetivo). En el marco de este TFM, esta técnica adquiere una dimensión crítica: el procesamiento de lenguas minorizadas con recursos computacionales limitados.

2.3.1. Inferencia multilingüe y pesos compartidos

Los Modelos de Lenguaje de Gran Tamaño (LLMs) han sido entrenados sobre corpus masivos que incluyen cientos de idiomas. Durante este proceso, las redes neuronales identifican estructuras universales de la lengua (como la relación entre sujetos, verbos y complementos). A través del algoritmo de retropropagación o backpropagation, el modelo ajusta sus pesos internos para minimizar el error de predicción. Conneau et al. (2020)

Cuando el sistema procesa una lengua de la familia maya, como el Kaqchikel, no parte de cero. Aprovecha la "memoria técnica" de estructuras gramaticales similares encontradas en otros idiomas. Este fenómeno permite que el aprendizaje de una lengua minoritaria ayude a otras; por ejemplo, la comprensión de la morfología aglutinante o el uso de glosas en el K'iche' prepara los circuitos del modelo para ser mucho más eficiente al enfrentarse al Mam o, por extensión, a lenguas no relacionadas pero igualmente complejas (como el quechua o el ainu).

2.3.2. El valor de los datos estructurados (XML/LaTeX)

La mayor barrera para la digitalización de lenguas minoritarias es la "escasez de datos" (Data Scarcity). Sin embargo, el sistema desarrollado en este trabajo soluciona este

problema mediante la generación de contenido de alta fidelidad. Aceleración del Fine-Tuning: Los archivos XML y LaTeX que genera la aplicación constituyen lo que en IA llamamos "Gold Standard." datos de referencia. Con una cantidad mínima de estas páginas estructuradas, es posible realizar un ajuste fino (fine-tuning) de modelos más pequeños y locales. Eficiencia en pocos ejemplos (Few-shot Learning): Como se observa en la arquitectura de carpetas de este proyecto (módulo data/few-shot), el modelo es capaz de aprender la jerarquía de una gramática completa tras ver solo dos o tres ejemplos bien etiquetados. Esta capacidad de generalización es extrapolable a cualquier lengua del mundo: si el pipeline es robusto para la complejidad de las lenguas mayas, su aplicación a otras familias lingüísticas requiere una inversión de tiempo y datos drásticamente inferior.

2.3.3. Democratización del conocimiento lingüístico

La metodología propuesta en este TFM establece un precedente para la preservación del patrimonio digital global. Al transformar "fotos de libros." en "datos estructurados validados", estamos creando el combustible necesario para que los algoritmos de back-propagation de futuros modelos comprendan lenguas que, de otro modo, quedarían fuera del radar tecnológico. En conclusión, el Transfer Learning permite que el esfuerzo dedicado a digitalizar la Academia de Lenguas Mayas de Guatemala se convierta en un activo científico universal. La estructura lógica aprendida y los mecanismos de corrección implementados (xml_corrector.py) funcionan como un esquema transferible que garantiza que, con muy poca información inicial, cualquier lengua minorizada pueda alcanzar el mismo nivel de accesibilidad digital que las lenguas mayoritarias.

El desarrollo de sistemas de gestión y monitorización de documentos es un área de gran relevancia en el ámbito empresarial, especialmente en sectores donde la integridad y disponibilidad de la información son críticas. En este capítulo, se analiza el impacto de la pérdida de documentos, las iniciativas existentes para su monitorización y las tecnologías web más utilizadas en la actualidad.

Aquí tienes la sección ampliada con los cambios solicitados, incorporando mayor detalle y referencias bibliográficas:

2.4. Impacto y costes de la pérdida de documentos

La gestión documental efectiva constituye un pilar fundamental en la operativa empresarial moderna, donde la digitalización y el volumen de información generada plantean desafíos crecientes. Una gestión inadecuada puede desencadenar consecuencias devastadoras tanto a nivel operativo como legal y reputacional.

2.4.1. Riesgos en el ciclo de vida documental

El proceso de gestión de documentos implica múltiples etapas críticas donde pueden producirse fallos. Según «ISO/TR 15801:2017 Document management – Information stored electronically – Recommendations for trustworthiness and reliability», hasta el 43 %

de las pérdidas documentales ocurren durante las fases de transferencia o almacenamiento temporal. Un riesgo particular surge cuando la generación de documentos se delega a proveedores externos, práctica común en sectores como el legal. Este outsourcing introduce vulnerabilidades al:

- Multiplicar los puntos de contacto en el flujo documental
- Requerir protocolos de transferencia seguros entre sistemas heterogéneos
- Dificultar la auditoría de trazabilidad completa

2.4.2. Consecuencias operativas y regulatorias

En el ámbito empresarial, la falta de acceso a información crítica genera interrupciones en cadena. Los costes asociados, como detalla el informe *Coste de una filtración de datos 2024*, incluyen:

- Costes directos de recuperación (€35.000-€250.000 por incidente)
- Multas regulatorias (hasta 4 % del volumen de negocio anual bajo GDPR)
- Litigios por incumplimiento contractual

Sectores regulados como el bancario o sanitario enfrentan riesgos amplificadas. La pérdida de documentos que contengan datos personales compromete no solo la privacidad, sino que genera responsabilidades legales escalonadas. El análisis de Ripcord Team (2020) revela que el 23 % de las brechas en estos sectores se originan en fallos de gestión documental, con costes medios por registro comprometido entre €350-€700.

2.5. Otras iniciativas

Ante la necesidad de garantizar la disponibilidad y seguridad de los documentos, se han desarrollado diversas iniciativas para su monitorización y gestión. Entre las más comunes se encuentran:

- **Soluciones Open Source** : Dos de las aplicaciones Open Source más conocidas para la gestión de documentos son Alfresco y OpenKM. Alfresco es un sistema de gestión documental open source desarrollado en Java. Su versión Community es gratuita pero requiere especialistas para su implementación. Aunque ofrece potentes APIs de integración, su versión Enterprise tiene costos prohibitivos para medianas empresas (Evolvia, 2023). Por otro lado cabe destacar que las soluciones open source preexistentes requieren conocimientos técnicos avanzados para su implementación y mantenimiento.

Por otro lado, OpenKM es una plataforma libre con interfaz web basada en tecnologías Java. Su versión Community sigue el modelo GPLv2, pero las versiones Professional y Cloud son comerciales. Aunque ofrece búsqueda avanzada y flujos de trabajo, su escalabilidad en entornos complejos es limitada (OpenKM, 2023).

- Plataformas cloud comerciales (OneDrive/Google Drive): Soluciones SaaS fáciles de implementar pero con limitaciones en control de datos, cumplimiento normativo para sectores regulados y dependencia del proveedor. Carecen de funcionalidades avanzadas de gestión documental como workflows complejos o metadata estructurada. La principal desventaja de las plataformas cloud comerciales es que carecen de control sobre los datos y no cumplen con regulaciones estrictas.
- Sistemas de monitorización mediante scripts: Soluciones basadas en scripts personalizados que verifican el estado de los documentos en una base de datos. Aunque son flexibles y de bajo coste, su mantenimiento puede ser complejo y no suelen escalar bien en entornos con grandes volúmenes de datos.
- Bases de datos duplicadas: Estrategia que consiste en mantener copias redundantes de los documentos en diferentes servidores. Aunque mejora la disponibilidad, implica un mayor coste de almacenamiento y puede complicar la sincronización entre las bases de datos.
- Consultas SQL básicas: Uso de consultas Structured Query Language (SQL) para verificar el estado de los documentos. Esta aproximación es sencilla pero limitada, ya que no ofrece funcionalidades avanzadas como la generación de informes o la integración con otras herramientas.

Estas iniciativas, aunque útiles en ciertos contextos, presentan limitaciones en términos de escalabilidad, seguridad y facilidad de uso, lo que justifica la necesidad de soluciones más avanzadas como SIDManager.

2.6. Tecnologías web

El desarrollo de aplicaciones web modernas requiere la selección de tecnologías adecuadas que permitan construir sistemas eficientes, seguros y escalables.

En este contexto, conviene distinguir entre un script y una aplicación web. Un *script* es un conjunto de instrucciones escritas en un lenguaje de programación (como Python o bash) que se ejecutan de manera secuencial para realizar tareas específicas. Aunque son útiles para automatizar procesos, los scripts suelen carecer de una interfaz de usuario y no están diseñados para manejar grandes volúmenes de datos o usuarios concurrentes.

En cambio, una *aplicación web* (Amazon, 2025) es un sistema completo que incluye una interfaz de usuario, lógica de negocio y persistencia de datos. Las aplicaciones web están diseñadas para ser accesibles desde navegadores y suelen utilizar tecnologías como HyperText Markup Language (HTML), Cascading Style Sheets (CSS), JavaScript en el front-end, y frameworks como Spring Boot o Django en el backend. A diferencia de los scripts, las aplicaciones web son escalables, seguras y ofrecen una experiencia de usuario más completa. En este capítulo se analizan las tecnologías más relevantes en este último ámbito.

2.6.1. Metodos HTTP

Como sostiene la Mozilla Developer Network (2025), los métodos HTTP (HyperText Transfer Protocol) son la base de la comunicación en la web y definen las operaciones que se pueden realizar sobre los recursos. Los más utilizados son los métodos GET y los métodos POST.

Un método GET se utiliza para solicitar datos de un recurso específico. Es idempotente, lo que significa que múltiples solicitudes GET no modifican el estado del servidor. Este método es ideal para recuperar información, como listados de documentos o detalles de un usuario.

Por contra, un método POST se utiliza para enviar datos al servidor, generalmente para crear nuevos recursos. A diferencia de GET, POST no es idempotente, ya que cada solicitud puede generar un cambio en el servidor. Este método es común en formularios de registro o envío de datos.

2.6.2. API

Una Application Programming Interface (API) es un conjunto de reglas y protocolos que permite la comunicación entre diferentes sistemas. Las APIs son fundamentales en el desarrollo de aplicaciones modernas, ya que facilitan la integración entre servicios y permiten construir sistemas modulares y escalables.

2.6.3. REST

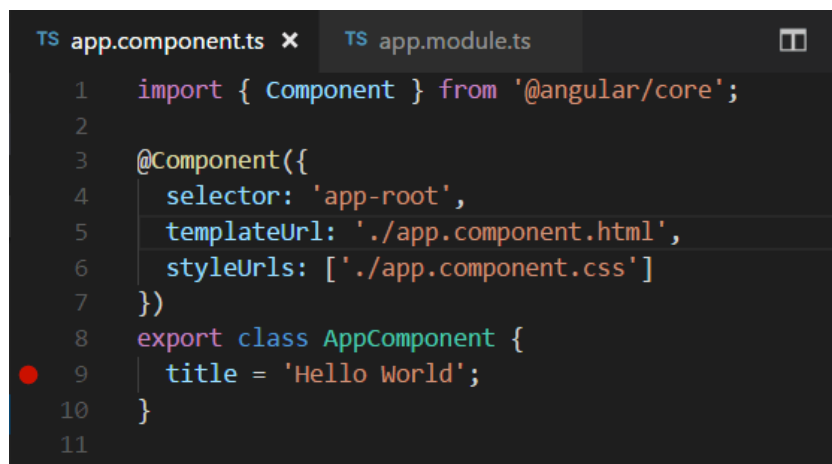
Representational State Transfer (REST) es un estilo arquitectónico para diseñar APIs web. Se basa en el uso de métodos HTTP (GET, POST, PUT, DELETE) para realizar operaciones sobre recursos identificados por URLs. REST es ampliamente utilizado debido a su simplicidad, escalabilidad y compatibilidad con diferentes tecnologías. Además, las APIs RESTful son fáciles de consumir desde clientes web, móviles o de escritorio, lo que las convierte en una opción ideal para aplicaciones como SIDManager.

2.7. Herramientas para el desarrollo

En este capítulo se describen las principales tecnologías y herramientas utilizadas en el desarrollo de esta aplicación web, analizando su origen, propósito y cómo contribuyen al proyecto.

2.7.1. Angular

Tal y como explica Żurawski (2024), Angular es un framework de desarrollo web creado por Google en 2010, diseñado para facilitar la construcción de aplicaciones de una sola página (SPA) en el lado del cliente. Estaba basado en JavaScript, así se ve en JavaScript definition de Mozilla Developer Network pero desde la versión 2, tanto internamente en la aplicación como para hacer desarrollos con esta, se utiliza TypeScript (TypeScript

A screenshot of a code editor with a dark theme. The editor has two tabs at the top: 'TS app.component.ts' (active) and 'TS app.module.ts'. The code in the active tab is as follows:

```
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
9    title = 'Hello World';
10 }
11
```

A red dot is visible on the left margin next to line 9.

Figura 2.1.: Ejemplo de componentes de Angular con código Typescript.

Language Specification, 2013) un superconjunto de JavaScript que añade tipado estático, mejorando la robustez y escalabilidad de las aplicaciones desarrolladas con Angular. El principal propósito de Angular es simplificar la creación de interfaces de usuario dinámicas y reactivas, permitiendo que los desarrolladores gestionen el contenido de forma más eficiente. Su arquitectura basada en componentes y el uso de un sistema de inyección de dependencias hacen que Angular sea muy adecuado para grandes aplicaciones front-end. Además, ofrece un sistema de dos vías de enlace de datos (two-way data binding) que sincroniza de manera automática los cambios entre el modelo y la vista.

Angular es parte de un conjunto más amplio de herramientas y tecnologías de Google, por lo que su integración con otros servicios de Google es fluida. A lo largo de los años, ha ganado popularidad en el desarrollo front-end, especialmente en entornos empresariales, debido a su capacidad para manejar aplicaciones de gran escala y su enfoque en la modularidad y reutilización de código.

Uno de los lenguajes que se utiliza en Angular es TypeScript, que proporciona características de programación orientada a objetos como clases, interfaces y herencia, facilitando el desarrollo de aplicaciones más mantenibles. Este lenguaje lo hace ideal para proyectos de gran tamaño, donde el control de tipos puede ayudar a reducir errores. A pesar de ser un framework basado en el front-end, Angular puede integrarse con varios backends, como Node.js, Django, Spring, entre otros.

Angular es un framework respaldado por Google, lo que asegura un ciclo de vida estable y una comunidad activa que contribuye a su mejora continua. Además, dado que Angular es open-source, la comunidad también juega un papel clave en su evolución, lo que garantiza que el framework continúe siendo relevante en la industria del desarrollo web.

Aunque Angular es muy popular, su curva de aprendizaje puede ser algo pronunciada, especialmente para desarrolladores novatos. Además, debido a su carácter completo, puede ser excesivo para aplicaciones más pequeñas, donde un framework más ligero podría

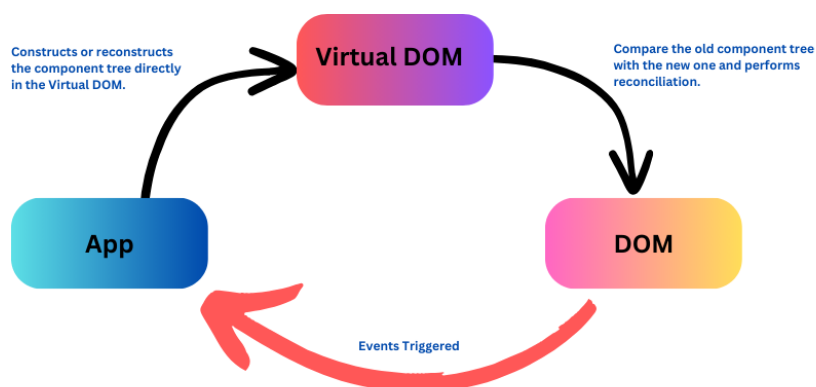


Figura 2.2.: Ejemplo de DOM virtual de React hecho por Angular Minds (2024)

ser suficiente. Sin embargo, en proyectos de gran escala que requieren una estructura bien organizada y mantenible, Angular sigue siendo una opción sólida.

2.7.2. React

React es una biblioteca de JavaScript para la construcción de interfaces de usuario, desarrollada por Facebook (ahora Meta), como se puede apreciar en el artículo Facebook social network, y lanzada al público en 2013. Su principal propósito es facilitar el desarrollo de aplicaciones web interactivas y reactivas mediante un enfoque basado en componentes reutilizables.

React se enfoca en la parte de la vista (view) en el patrón de diseño Model-View-Controller (MVC) como explica Reenskaug y Coplien en el estudio The DCI Architecture: A New Vision of Object-Oriented Programming, lo que permite crear interfaces dinámicas sin tener que recargar la página web.

Una de las características más destacadas de React es su uso del Virtual Document Object Model (VDOM) tal y como explica en Understanding the Power of Virtual DOM in Web Development. Este artículo sostiene que se obtiene una mejora significativa en el rendimiento al minimizar la manipulación directa del DOM del navegador. En lugar de modificar el DOM completo con cada cambio, React crea una copia de este en memoria y solo actualiza las partes que han cambiado, lo que hace que las aplicaciones sean más rápidas y eficientes.

React está escrito en JavaScript, pero con soporte extendido para JavaScript XML (JSX) (para más información, ver el informe de geeksforgeeks llamado React JSX) una extensión de sintaxis que permite escribir elementos de la interfaz de usuario de forma declarativa, como si fueran parte del código HTML. Esto facilita a los desarrolladores visualizar claramente la estructura del componente dentro del código JavaScript, simplificando el desarrollo y mantenimiento de la interfaz.

El ecosistema de React ha crecido enormemente desde su lanzamiento. Aunque originalmente fue concebido para aplicaciones de una sola página (SPA), React ha evolu-

cionado para ser usado tanto en el desarrollo front-end como en aplicaciones móviles, mediante el uso de React Native. Esto convierte a React en una tecnología versátil, usada en una amplia variedad de plataformas.

Una ventaja importante de React es su flexibilidad: no impone una arquitectura rígida y se puede integrar fácilmente con otras bibliotecas o frameworks.

A pesar de sus ventajas, React tiene algunas desventajas. Una de ellas es que, aunque es fácil de aprender para pequeños proyectos, gestionar el estado en aplicaciones grandes puede volverse complicado, lo que requiere el uso de bibliotecas externas como Redux o Context API. Además, React es solo una biblioteca para la vista, lo que significa que carece de muchas funcionalidades de un framework completo, como el enrutamiento o la gestión de dependencias, que deben añadirse manualmente.

React ha ganado una enorme popularidad desde su lanzamiento y sigue siendo una de las bibliotecas más utilizadas para el desarrollo web, debido a su eficiencia, flexibilidad y soporte por parte de una comunidad muy activa.

2.7.3. Spring

Spring es un marco de trabajo (framework) para el desarrollo de aplicaciones en Java, como sostiene Journal of Computer Sciences Institute, 2024 , ampliamente utilizado en entornos empresariales debido a su flexibilidad y capacidad para facilitar el desarrollo de aplicaciones robustas y escalables. Fue desarrollado por Rod Johnson y lanzado inicialmente en 2003, como una alternativa ligera a los frameworks Java más pesados de la época. Spring permite construir aplicaciones que siguen los principios de inyección de dependencias y programación orientada a aspectos (para más detalle sobre programación orientada a objetos ver Kiczales (2024)) , lo que mejora la modularidad y facilita la gestión de la complejidad en proyectos grandes.

El propósito principal de Spring es simplificar el desarrollo de aplicaciones en Java, proporcionando una infraestructura sólida que resuelve los problemas comunes en la creación de aplicaciones de servidor. Además de su flexibilidad, Spring se destaca por sus capacidades de integración con otros frameworks y tecnologías, lo que lo convierte en una solución versátil para la creación de aplicaciones web, Application Programming Interface (API) y microservicios.

Spring es el framework base que se extiende a través de módulos como Spring MVC (para aplicaciones web), Spring Security (para control de acceso) y Spring Data (para gestión de bases de datos). Gracias a su popularidad, sigue siendo el framework Java más utilizado para el desarrollo de aplicaciones empresariales.

2.7.4. Spring Boot

Spring Boot es una extensión del framework Spring que fue publicada en 2014 por Pivotal Software. Está diseñada para simplificar el proceso de configuración y arranque de las aplicaciones desarrolladas en Spring. Mientras que Spring ofrece una gran flexibilidad, su configuración puede ser tediosa y propensa a errores. Aquí es donde Spring Boot aporta su valor: reduce la necesidad de configuración manual gracias a las convenciones

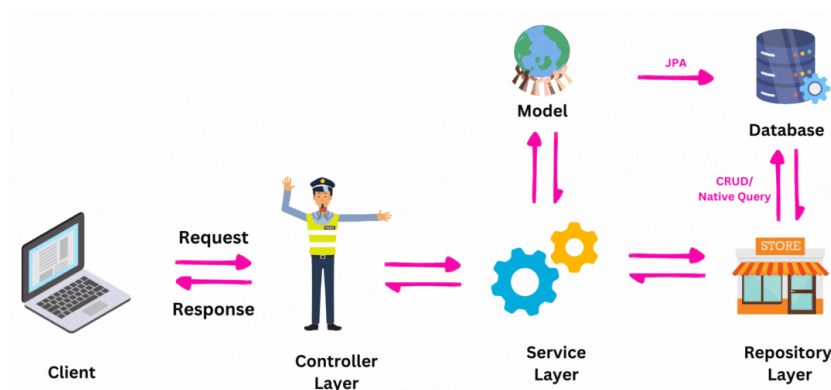


Figura 2.3.: Capas de Spring Boot hecho por Fadatare (2025)

predeterminadas y a una serie de herramientas integradas como Java Persistence API (JPA) (que explicaremos más adelante), Spring Web (Faber, 2020) y muchas más.

Spring Boot permite a los desarrolladores centrarse en la lógica del negocio en lugar de en la configuración del entorno, ya que automatiza la mayor parte de la configuración, incluyendo el servidor web, la integración con bases de datos y la seguridad. Esto es posible gracias a la división en las capas: vista (view) o cliente (client), controlador (controller), servicio (service), modelo (model) y repositorio (repository) entre otras, como se ve en la figura 2.3. Esto lo convierte en una excelente opción para la creación rápida de prototipos, aplicaciones de producción listas para el despliegue y microservicios. Su popularidad ha crecido rápidamente debido a la eficiencia que proporciona, lo que ha hecho que muchos desarrolladores lo elijan para proyectos grandes y pequeños.

Spring Boot se integra fácilmente con otros componentes de Spring, como Spring Security y Spring Data, y se basa en las mismas buenas prácticas que han hecho de Spring un estándar en la industria del desarrollo de software. Por ejemplo, Spring Boot facilita la integración con JPA, el cual permite la persistencia de datos en bases de datos relacionales.

2.7.5. JPA

JPA (Java Persistence API) es una especificación que simplifica la interacción con bases de datos en aplicaciones Java. Fue introducida en 2006 como parte de la plataforma Java Enterprise Edition (Java EE) 5 y desarrollada por Oracle. Su objetivo es facilitar el mapeo de objetos Java a tablas de bases de datos relacionales sin la necesidad de escribir SQL manualmente. Con JPA, los desarrolladores pueden centrarse en el código de negocio sin preocuparse por los detalles de la persistencia, como se ve en el artículo de Fowler y Sadalage (2011).

Una de las grandes ventajas de JPA es que reduce la cantidad de código repetitivo necesario para realizar operaciones Create, Read, Update, Delete (CRUD) en la base de datos.

JPA permite que los desarrolladores trabajen con objetos de dominio directamente, delegando la gestión de las transacciones y el acceso a la base de datos en el framework subyacente. En combinación con Spring Data JPA, la interacción con la base de datos se hace más sencilla y eficiente, eliminando la necesidad de escribir consultas SQL explícitas en la mayoría de los casos.

JPA se utiliza principalmente en aplicaciones que requieren una gestión compleja de datos, y su integración con Spring Boot a través de Spring Data JPA permite acelerar el desarrollo al proporcionar un enfoque estandarizado para la persistencia de datos.

2.7.6. Maven

Maven es una herramienta de gestión de proyectos y de automatización de construcción para aplicaciones Java, desarrollada por la Apache Software Foundation y lanzada en 2004 (Apache Software Foundation, 2023).

Su principal función es gestionar las dependencias de un proyecto, es decir, todas las bibliotecas externas que el proyecto necesita para funcionar. Maven permite a los desarrolladores definir estas dependencias en un archivo eXtensible Markup Language (XML) llamado `pom.xml` y luego descarga automáticamente las versiones correctas de estas bibliotecas desde repositorios remotos.

Una de las mayores ventajas de Maven es su capacidad para estandarizar el proceso de construcción de aplicaciones Java, independientemente del entorno en el que se desarrolle. Esto asegura que los proyectos construidos con Maven sean consistentes y que se puedan replicar fácilmente en cualquier máquina de desarrollo.

En este proyecto, Maven juega un papel crucial al manejar las dependencias de Spring Boot, JPA y Thymeleaf, asegurando que todas las bibliotecas necesarias estén disponibles y configuradas correctamente.

2.7.7. Thymeleaf

Thymeleaf es un motor de plantillas para aplicaciones Java, creado por Daniel Fernández en 2011, que permite la generación dinámica de contenido HTML, XML y otros formatos de texto, tal y como lo explican sus creadores en la web oficial Thymeleaf Team (2023).

Thymeleaf se usa principalmente para construir la vista (view) o plantilla (template) en aplicaciones web, y es altamente compatible con Spring, ya que facilita la integración de los modelos de datos Java con las plantillas HTML de manera eficiente.

Una de las principales ventajas de Thymeleaf es que permite crear plantillas que pueden ser previsualizadas sin necesidad de ejecutar la aplicación en un servidor. Esto lo hace particularmente útil para el desarrollo colaborativo entre diseñadores y programadores, ya que los archivos HTML son completamente editables desde el lado del cliente.

Thymeleaf está diseñado para funcionar bien con Spring MVC y Spring Boot, lo que lo convierte en una opción natural para las aplicaciones web que necesitan generar contenido dinámico basado en datos proporcionados por el servidor.

2.8. Ciberseguridad

La ciberseguridad es un aspecto crítico en el desarrollo de aplicaciones modernas, especialmente cuando se manejan datos sensibles como credenciales de usuarios, información personal, contraseñas y documentos privados. En este apartado, se describen las tecnologías y prácticas más relevantes en el ámbito de la ciberseguridad, con un enfoque en el cifrado de datos y las capas de seguridad en el proceso de autenticación.

El cifrado es una técnica fundamental para proteger la confidencialidad e integridad de los datos. Consiste en transformar la información en un formato ilegible para cualquier persona que no posea la clave de descifrado adecuada. A continuación, se describen los principales enfoques y tecnologías utilizadas en el cifrado de datos.

2.8.1. Cifrado simétrico

El cifrado simétrico (Schneier, 1996) utiliza una única clave para cifrar y descifrar los datos. Este enfoque es eficiente en términos de rendimiento, ya que requiere menos recursos computacionales en comparación con el cifrado asimétrico. Sin embargo, la gestión segura de la clave es un desafío, ya que debe compartirse entre las partes que necesitan cifrar y descifrar los datos.

AES (*Advanced Encryption Standard*) es uno de los algoritmos de cifrado simétrico más utilizados y seguros. AES opera en bloques de 128 bits y admite claves de 128, 192 o 256 bits. Su fortaleza radica en su resistencia a ataques criptográficos y su eficiencia en términos de velocidad.

AES puede utilizarse en diferentes modos de operación, como Electronic Codebook (ECB), Cipher Block Chaining (CBC) y Galois/Counter Mode (GCM). Cada modo tiene sus propias características y niveles de seguridad.

2.8.2. Cifrado asimétrico

El cifrado asimétrico, también conocido como cifrado de clave pública, utiliza un par de claves: una clave pública para cifrar los datos y una clave privada para descifrarlos. Este enfoque es ideal para escenarios donde la clave no puede compartirse de manera segura, como en la comunicación entre dos partes que no han establecido previamente una relación de confianza.

Rivest–Shamir–Adleman (Cryptosystem) (RSA) es uno de los algoritmos de cifrado asimétrico más populares. RSA se basa en la dificultad de factorizar números grandes en sus componentes primos. Aunque es seguro, su rendimiento es inferior al cifrado simétrico, por lo que se utiliza principalmente para cifrar claves o datos pequeños.

Curvas Elípticas Elliptic Curve Cryptography (ECC) es una alternativa más moderna y eficiente a RSA. ECC ofrece un nivel de seguridad similar con claves más cortas, lo que reduce el consumo de recursos y mejora el rendimiento.

2.8.3. Cifrado de contraseñas

El cifrado de contraseñas es un caso especial donde se utilizan funciones de hash unidireccionales para almacenar las contraseñas de manera segura. A diferencia del cifrado tradicional, el hash no puede revertirse, lo que garantiza que incluso si la base de datos es comprometida, las contraseñas no puedan recuperarse.

- BCrypt: Es una función de hash diseñada específicamente para contraseñas. BCrypt utiliza un valor de *salt* (Hughes, 2024) único para cada contraseña, lo que aumenta la seguridad frente a ataques de diccionario y fuerza bruta. Además, su diseño permite ajustar la complejidad del hashing, lo que lo hace resistente a los avances en hardware.
- Argon2: Es una función de hash moderna que ha ganado popularidad debido a su resistencia a ataques de fuerza bruta y su capacidad para utilizar múltiples recursos del sistema (CPU, memoria y paralelismo). Argon2 es considerado uno de los algoritmos más seguros para el cifrado de contraseñas.

2.8.4. Capas de Seguridad en el login

El proceso de autenticación es uno de los puntos más vulnerables en cualquier aplicación, ya que es el primer paso para acceder a los recursos protegidos. Para garantizar la seguridad del login, es esencial implementar múltiples capas de seguridad que protejan contra ataques comunes y refuercen la integridad del sistema.

Los ataques de fuerza bruta intentan adivinar las credenciales de un usuario probando múltiples combinaciones de nombres de usuario y contraseñas. Para proteger contra estos ataques, es esencial implementar medidas como:

- Límite de Intentos: Bloquear temporalmente la cuenta después de un número determinado de intentos fallidos. Esto dificulta que un atacante pueda probar múltiples combinaciones en un corto período de tiempo.
- CAPTCHA: Requerir que el usuario resuelva un desafío CAPTCHA después de varios intentos fallidos. Esto ayuda a distinguir entre usuarios legítimos y bots automatizados.

Los ataques Cross-Site Request Forgery (CSRF) explotan la confianza que un sitio web tiene en el navegador del usuario para realizar acciones no autorizadas. Para prevenir estos ataques, se pueden implementar las siguientes medidas:

- Tokens CSRF: Generar un token único para cada sesión de usuario y validarlo en cada solicitud. Esto garantiza que la solicitud proviene de una fuente legítima.
- SameSite Cookies: Configurar las cookies con el atributo SameSite para evitar que se envíen en solicitudes cruzadas entre sitios.

El registro y monitoreo de la actividad del usuario es esencial para detectar y responder a posibles amenazas de seguridad. Algunas prácticas recomendadas incluyen:

- Registro de Accesos: Registrar todos los intentos de login, tanto exitosos como fallidos, para identificar patrones sospechosos.
- Alertas de Seguridad: Configurar alertas para notificar a los administradores en caso de actividades inusuales, como múltiples intentos fallidos de login desde diferentes ubicaciones.

2.9. Diseño software

El diseño de software es una fase crítica en el desarrollo de aplicaciones, ya que define la estructura, organización y comportamiento del sistema. Un buen diseño no solo garantiza que el software cumpla con los requisitos funcionales, sino que también facilita su mantenimiento, escalabilidad y reutilización. En este apartado, se abordarán los patrones de diseño más comunes y los diagramas Unified Modeling Language (UML), herramientas esenciales para modelar y documentar sistemas de software.

2.9.1. Patrones de diseño software

Los patrones de diseño son soluciones probadas y reutilizables para problemas comunes en el desarrollo de software. Estos patrones proporcionan un enfoque estructurado para resolver desafíos específicos, mejorando la calidad del código y facilitando su mantenimiento. Para más información se puede obtener el libro de Erich Gamma y Vlissides (1994). A continuación, se describen algunos de los patrones más utilizados:

El patrón Fachada es un patrón estructural que proporciona una interfaz simplificada para un conjunto de interfaces en un subsistema. Este patrón es útil cuando un sistema es complejo y tiene múltiples componentes, ya que oculta la complejidad detrás de una interfaz única y fácil de usar. Por ejemplo, en una aplicación con múltiples servicios y bibliotecas, una fachada puede actuar como un punto de entrada único, reduciendo la dependencia entre los componentes y facilitando su uso.

El patrón Singleton es un patrón creacional que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. Este patrón es útil cuando se necesita un único punto de control, como en la gestión de configuraciones, conexiones a bases de datos o registros de logs. Al garantizar que solo exista una instancia, se evita la duplicación de recursos y se asegura un comportamiento consistente en toda la aplicación.

Además de los mencionados, existen otros patrones ampliamente utilizados, como:

- Patrón Observador (Observer): Permite que un objeto notifique a otros objetos sobre cambios en su estado.
- Patrón Estrategia (Strategy): Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables.

- Patrón Inyección de Dependencias (Dependency Injection): Facilita la gestión de dependencias entre componentes, promoviendo un código más modular y testeable.

2.9.2. Diagramas UML

El Lenguaje Unificado de Modelado (UML, por sus siglas en inglés) es un estándar utilizado para visualizar, especificar, construir y documentar sistemas de software. Los diagramas UML son una herramienta esencial para comunicar el diseño del software entre desarrolladores, analistas y stakeholders. A continuación, se describen los tipos de diagramas más relevantes.

Muchos artículos hablan del diagrama de secuencia UML como Li et al. (2004). Estos explican que el diagrama de secuencia es un tipo de diagrama de interacción que muestra cómo los objetos interactúan en un escenario específico a lo largo del tiempo. Este diagrama es especialmente útil para modelar el flujo de mensajes entre objetos en un sistema, lo que permite entender cómo se ejecutan las operaciones y en qué orden.

- Actores: Representan los roles externos que interactúan con el sistema, como usuarios o sistemas externos.
- Objetos: Representan las instancias de clases o componentes del sistema.
- Mensajes: Representan las interacciones entre objetos, mostrando el flujo de control y datos.
- Línea de vida: Muestra la existencia de un objeto durante el tiempo de ejecución.

Los diagramas de secuencia son ideales para modelar casos de uso complejos, como procesos de autenticación, transacciones o flujos de trabajo. Además, son una herramienta valiosa para identificar cuellos de botella y optimizar el rendimiento del sistema.

Además del diagrama de secuencia, UML incluye otros tipos de diagramas, como:

- Diagrama de clases: Muestra la estructura estática del sistema, incluyendo clases, atributos, métodos y relaciones.
- Diagrama de casos de uso: Representa las interacciones entre los actores y el sistema, describiendo las funcionalidades desde la perspectiva del usuario.
- Diagrama de actividades: Modela el flujo de trabajo o procesos de negocio, mostrando las actividades y decisiones involucradas.
- Diagrama de componentes: Muestra la organización física del sistema, incluyendo módulos, bibliotecas y dependencias.

En resumen, los patrones de diseño y los diagramas UML son herramientas fundamentales en el desarrollo de software moderno. Los patrones de diseño proporcionan soluciones reutilizables para problemas comunes, mientras que los diagramas UML facilitan la comunicación y documentación del diseño del sistema. Juntos, permiten crear aplicaciones robustas, mantenibles y escalables.

Personalmente, este proyecto ha sido una oportunidad para profundizar en tecnologías y conceptos que no formaban parte de mi conocimiento previo. Desde el aprendizaje de nuevas herramientas como Spring Boot y Maven hasta la adopción de buenas prácticas como la documentación con Javadoc y el uso de estándares de Java, he adquirido habilidades que serán valiosas en futuros proyectos. Además, la experiencia de trabajar en un proyecto de esta envergadura ha reforzado mi capacidad para planificar, organizar y resolver problemas de manera eficiente.

En conclusión, este proyecto no solo ha cumplido con sus objetivos iniciales, sino que también ha sentado las bases para futuras mejoras y desarrollos. Espero que el trabajo realizado contribuya al avance de aplicaciones similares y sirva como referencia para otros desarrolladores e investigadores.

Bibliografía

- Amazon. (2025). ¿Qué es una aplicación web? Consultado el 20 de marzo de 2025, desde <https://aws.amazon.com/es/what-is/web-application/>
- Angular Minds. (2024). How Virtual DOM Works In React. Consultado el 20 de marzo de 2025, desde <https://www.angularminds.com/blog/how-virtual-dom-works>
- Apache Software Foundation. (2023). Maven: Welcome to Apache Maven [Official documentation and overview of Apache Maven.]. Consultado el 20 de marzo de 2025, desde <https://maven.apache.org/>
- Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised Cross-lingual Representation Learning at Scale. En D. Jurafsky, J. Chai, N. Schlueter & J. Tetreault (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440-8451). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.747>
- Ding, Y., Luo, S., Dai, Y., Jiang, Y., Li, Z., Sun, Q., Martin, G., Liu, W., & Peng, Y. (2026). A Survey on MLLM-based Visually Rich Document Understanding: Methods, Challenges, and Emerging Trends. <https://arxiv.org/abs/2507.09861>
- Erich Gamma, R. J., Richard Helm, & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. *Addison-Wesley*, 1-35. Consultado el 20 de marzo de 2025, desde <https://www.pearson.com/us/higher-education/program/Gamma-Design-Patterns-Elements-of-Reusable-Object-Oriented-Software/PGM14333.html>
- Evoltia. (2023). Alfresco vs SharePoint: Comparativa de sistemas de gestión documental. Consultado el 15 de noviembre de 2023, desde <https://www.evoltia.com/en/alfresco-sharepoint-gestion-documental>
- Faber, J. (2020). Introducción a Spring Security. Consultado el 20 de marzo de 2025, desde <https://adictosaltrabajo.com/2020/05/21/introduccion-a-spring-security>
- Fadatare, R. (2025). Spring Boot Architecture: Controller, Service, Repository, Database & Architecture Flow. Consultado el 20 de marzo de 2025, desde <https://rameshfadatare.medium.com/spring-boot-architecture-controller-service-repository-database-architecture-flow-9144084818b0>
- Fowler, M., & Sadalage, P. (2011). Evolutionary Database Design. *ThoughtWorks Articles*. Consultado el 20 de marzo de 2025, desde <https://www.martinfowler.com/articles/evodb.html>
- geeksforgeeks. (2023). React JSX. Consultado el 20 de marzo de 2025, desde <https://www.geeksforgeeks.org/reactjs-jsx-introduction/>
- Hall, M. (s.f.). Facebook social network. Consultado el 20 de marzo de 2025, desde <https://www.britannica.com/money/Facebook>

- Hughes, A. (2024). Encryption vs. Hashing vs. Salting - What's the Difference? <https://www.pingidentity.com/en/resources/blog/post/encryption-vs-hashing-vs-salting.html>
- IBM. (2024). *Coste de una filtración de datos 2024*. IBM. <https://www.ibm.com/es-es/reports/data-breach>
- ISO/TR 15801:2017 *Document management – Information stored electronically – Recommendations for trustworthiness and reliability*. (2017). International Organization for Standardization. <https://www.iso.org/standard/66344.html>
- Mozilla Developer Network. (s.f.). JavaScript definition. Consultado el 20 de marzo de 2025, desde <https://developer.mozilla.org/es/docs/Web/JavaScript>
- Journal of Computer Sciences Institute. (2024). Examination of the performance and scalability of a web application in a reactive and imperative approach using the Spring Framework. Consultado el 20 de marzo de 2025, desde https://www.researchgate.net/publication/384478968_Examination_of_the_performance_and_scalability_of_a_web_application_in_a_reactive_and_imperative_approach_using_the_Spring_Framework
- Kiczales, G. (2024). Aspect Oriented Programming. Consultado el 20 de marzo de 2025, desde <https://www.cs.ubc.ca/~gregor/>
- Li, X., Liu, Z., & Jifeng, H. (2004). A formal semantics of UML sequence diagram. <https://doi.org/10.1109/ASWEC.2004.1290469>
- Make Computer Science Great Again. (2023). Understanding the Power of Virtual DOM in Web Development. Consultado el 20 de marzo de 2025, desde <https://medium.com/@MakeComputerScienceGreatAgain/understanding-the-power-of-virtual-dom-in-web-development-f31d7ac28b29>
- Mozilla Developer Network. (2025). Métodos de petición HTTP. Consultado el 20 de marzo de 2025, desde <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>
- OpenKM. (2023). Sistema de Gestión Documental OpenKM. Consultado el 15 de noviembre de 2023, desde <https://www.openkm.com/es/>
- Reenskaug, T., & Coplien, J. O. (2009). The DCI Architecture: A New Vision of Object-Oriented Programming. Consultado el 20 de marzo de 2025, desde <https://www.artima.com/articles/the-dci-architecture-a-new-vision-of-object-oriented-programming>
- Ripcord Team. (2020). *The True Cost of Poor Document Management*. Business Process Enablement. <https://blog.ripcord.com/resources/the-true-cost-of-poor-document-management>
- Schneier, B. (1996). Applied Cryptography: Protocols, Algorithms, and Source Code in C (2nd). Wiley, 45-78. Consultado el 20 de marzo de 2025, desde https://www.schneier.com/books/applied_cryptography/
- Shen, A., Chen, T., Sun, Y., Jiang, F., Feng, Y., Hu, K., & Liu, M. (2025). Vision-driven Adaptive Decoding for Document Understanding: A Dual-Path Visual-Language and OCR Framework with Prompt-aware Task Routing. *2025 9th International*

- Conference on Vision, Image and Signal Processing (ICVISIP)*, 1-5. <https://doi.org/10.1109/ICVISIP68610.2025.11451708>
- Singh, D., Bansal, A., & Bansal, N. (2015). A Review on Optical Character Recognition Using Various Techniques. <https://api.semanticscholar.org/CorpusID:201633108>
- Thymeleaf Team. (2023). Thymeleaf: Modern Server-Side Java Template Engine [Official documentation and overview of Thymeleaf.]. Consultado el 20 de marzo de 2025, desde <https://www.thymeleaf.org/>
- TypeScript Language Specification. (2013). Consultado el 20 de marzo de 2025, desde <https://web.archive.org/web/20131117065339/http://www.typescriptlang.org/Content/TypeScript%20Language%20Specification.pdf>
- Żurawski, P. (2024, mayo). Angular SSR: Your server-side rendering implementation guide. Consultado el 20 de marzo de 2025, desde <https://pretius.com/blog/angular-ssr/>

A. Anexo I. Esquema de base de datos

ClientUser	
PK	<u>email varchar(255)</u>
	password varchar(255) NOT NULL status int NOT NULL confirmationCode varchar(255) NOT NULL creation_date datetime NOT NULL confirmation_date datetime

Document	
PK	<u>client_app_id int</u> <u>client_doc_id int</u>
	provider_id varchar(255) NOT NULL received_at varchar(255) sent_at varchar(255)

Figura A.1.: Esquema de la base de datos

B. Anexo II. Código de la aplicación SIDManager

Listado B.1: Kaqchikel XML generado por el LLM a partir de la imagen de la página original

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <document>
3   <page number="203">
4     <heading>... / .. / .... Oración compleja</heading>
5     <line>La oración compleja se forma con más de una cláusula
6       , una de ellas es la principal y</line>
7     <line>otra la subordinada. La idea que expresan las clá
8       usulas subordinadas siempre dependen</line>
9     <line>de la cláusula principal, su posición es siempre
10      después de ésta. Una cláusula subordinada</line>
11     <line>se introduce por medio de subordinadores, aunque hay
12      excepciones. Son oraciones</line>
13     <line>complejas si dentro de la cláusula principal se
14      encuentran las siguientes cláusulas</line>
15     <line>subordinadas:</line>
16
17     <line>Cláusula relativa</line>
18     <line>Cláusula adjunta o adverbial</line>
19     <line>Cláusula de complemento</line>
20
21     <heading>... / .. / .... / . Clausula relativa</heading>
22     <line>La clausula relativa modifica a sustantivos,
23      describe alguna característica, por eso</line>
24     <line>funciona como adjetivo. La clausula relativa se
25      introduce por los subordinadores: ri,</line>
26     <line>re, la, achoq rik'in y achoq chi re o simplemente
27      subordinador. Estas particulas se</line>
28     <line>conocen con los nombres de pronombre relativo o
29      relativizador. Si modifica al sujeto u</line>
30     <line>objeto, se introduce con las particulas: ri, re, la.
31      Si modifica al instrumento o compania,</line>
32     <line>la clausula relativa se introducen con la frase
33      achoq rik'in; y si la clausula no se</line>
34     <line>introduce por subordinador, la clausula relativa se
35      escribe entre comas. Ejemplos:</line>
```

```

25
26
27 <line>Modificando al sujeto:</line>
28 <interlinear_gloss>
29   <parallel_phrase>
30     <original>Ri ix\{"{o}q, ri xkos, xb'eruloq'o' xkoya
31       '.</original>
32     <translation>La mujer que se cansó fue a comprar
33       tomate</translation>
34   </parallel_phrase>
35   <morpheme_analysis>
36     <unit>
37       <form>Ri</form>
38       <gloss>DET</gloss>
39     </unit>
40     <unit>
41       <form>ix\{"{o}q,</form>
42       <gloss>mujer</gloss>
43     </unit>
44     <unit>
45       <form>ri</form>
46       <gloss>CR</gloss>
47     </unit>
48     <unit>
49       <form>x-</form>
50       <gloss>COM-</gloss>
51     </unit>
52     <unit>
53       <form>\0}-</form>
54       <gloss>B3s-</gloss>
55     </unit>
56     <unit>
57       <form>kos,</form>
58       <gloss>cansar</gloss>
59     </unit>
60     <unit>
61       <form>x-</form>
62       <gloss>COM-</gloss>
63     </unit>
64     <unit>
65       <form>\0}-</form>
66       <gloss>B3s-</gloss>
67     </unit>
68     <unit>
69       <form>b'e-</form>
70       <gloss>MOV-</gloss>
71     </unit>
72     <unit>
73       <form>ru-</form>

```

```

72         <gloss>A3s-</gloss>
73     </unit>
74     <unit>
75         <form>loq'-</form>
76         <gloss>comprar-</gloss>
77     </unit>
78     <unit>
79         <form>o'</form>
80         <gloss>SC</gloss>
81     </unit>
82     <unit>
83         <form>xkoya'.</form>
84         <gloss>tomate</gloss>
85     </unit>
86 </morpheme_analysis>
87 <syntactic_analysis>
88     <unit>
89         <form>ri xkos</form>
90         <gloss>CR</gloss>
91     </unit>
92 </syntactic_analysis>
93 </interlinear_gloss>
94
95 <interlinear_gloss>
96     <parallel_phrase>
97         <original>Re ak'wal, re xtzaq, xoq'.</original>
98         <translation>Este niño que se cayó, lloró.</
99             translation>
100     </parallel_phrase>
101     <morpheme_analysis>
102         <unit>
103             <form>Re</form>
104             <gloss>DET</gloss>
105         </unit>
106         <unit>
107             <form>ak'wal,</form>
108             <gloss>niño,</gloss>
109         </unit>
110         <unit>
111             <form>re</form>
112             <gloss>CR</gloss>
113         </unit>
114         <unit>
115             <form>x-</form>
116             <gloss>COM-</gloss>
117         </unit>
118         <unit>
119             <form>{\0}-</form>

```

```

120         <gloss>B3s-</gloss>
121     </unit>
122     <unit>
123         <form>tzaq,</form>
124         <gloss>caer</gloss>
125     </unit>
126     <unit>
127         <form>xloq'.</form>
128         <gloss>llorar</gloss>
129     </unit>
130 </morpheme_analysis>
131 <syntactic_analysis>
132     <unit>
133         <form>re xtzaq</form>
134         <gloss>CR</gloss>
135     </unit>
136 </syntactic_analysis>
137 </interlinear_gloss>
138
139 <interlinear_gloss>
140     <parallel_phrase>
141         <original>La k'aj\"{o}l, la xloq'o ri wexaj, xb'e
142         .</original>
143         <translation>Ese muchacho que compr\'o el pantal\'
144         on, se fue.</translation>
145     </parallel_phrase>
146     <syntactic_analysis>
147         <unit>
148             <form>la xloq'o ri wexaj</form>
149             <gloss>CR</gloss>
150         </unit>
151     </syntactic_analysis>
152 </interlinear_gloss>
153 </page>
154 </document>
155 }

```

Listado B.2: Función getNotReceivedDocuments del controlador

```

1 @GetMapping ("/not-received")
2 public String getNotReceivedDocuments(Model model){
3     logger.debug(" ----- Entering getNotReceivedDocuments ----- "
4     );
5     model.addAttribute("documents" , documentService.
6     getNotReceivedDocuments());
7     model.addAttribute ("isShowingUnsaveds", "Yes");
8     logger.debug(" ----- Exiting getNotReceivedDocuments ----- ");
9 }

```



```
7     return "documentsView";
8 }
```

Listado B.3: Función getNotReceivedDocuments del servicio

```
1 public List<Document> getNotReceivedDocuments() {
2     logger.debug(" Entering getNotReceivedDocuments ");
3     return documentRepository.findByReceivedAtIsNull();
4 }
```

Listado B.4: Función listDocuments del controlador

```
1 @GetMapping("/list")
2 public String listDocuments(
3     @RequestParam(value = "min-date", required = false)
4         String startDate,
5     @RequestParam(value = "max-date", required = false) String
6         endDate,
7     Model model) throws Exception {
8     logger.debug(" ----- Entering listDocuments ----- ");
9
10    DateRange dateRange = documentService.checkDates(startDate,
11        endDate, model);
12
13    model.addAttribute("documents",
14        documentService.getFilteredDocuments(
15            dateRange.getStartDate(),
16            dateRange.getEndDate()
17        ));
18    model.addAttribute("startDate", dateRange.getStartDate());
19    model.addAttribute("endDate", dateRange.getEndDate());
20    model.addAttribute("isShowingUnsaveds", "No");
21
22    logger.debug(" ----- Exiting listDocuments ----- ");
23    logger.debug(" ----- ");
24
25    return "documentsView";
26 }
```

Listado B.5: Función getFilteredDocuments del servicio

```
1 public List<Document> getFilteredDocuments(Date startDate, Date
2     endDate) {
3     logger.debug(" Entering getFilteredDocuments ");
4     return documentRepository.findFilteredDocuments(startDate,
5         endDate);
6 }
```

Listado B.6: Función findFilteredDocuments del repositorio

```

1 @Query(value = "SELECT CLIENT_APP_ID, CLIENT_DOC_ID, PROVIDER_ID,
2   SENT_AT, RECEIVED_AT"
3   + " FROM documents WHERE SENT_AT > :startDate AND
4   SENT_AT < :endDate", nativeQuery = true)
5 List<Document> findFilteredDocuments(@Param("startDate") Date
6   startDate, @Param("endDate") Date endDate);

```

Listado B.7: Función checkDates del servicio

```

1 public DateRange checkDates(String stringStartDate, String
2   stringEndDate, Model model) {
3   final int daysRange = 10;
4   DateRange dateRange;
5   boolean isCorrectRange = true;
6
7   try {
8     if (DateUtils.checkDate(stringStartDate) && DateUtils.
9       checkDate(stringEndDate)) {
10      Date startDate = DateUtils.stringToUtilsDate(
11        stringStartDate);
12      Date endDate = DateUtils.stringToUtilsDate(
13        stringEndDate);
14
15      int daysBetween = DateUtils.daysBetween(startDate,
16        endDate);
17
18      if(daysBetween != -1 && daysBetween <= daysRange) {
19        dateRange = new DateRange(startDate, endDate);
20        logger.info(" Returning FILTERED documents ");
21      } else {
22        dateRange = new DateRange(-daysRange);
23
24        logger.warn(" Exceded maximum day range ( " +
25          daysBetween +
26          " ) with: " +
27          daysRange);
28        logger.warn(" Returning last 10 days ");
29
30        isCorrectRange = false;
31      }
32    } else if (DateUtils.checkDate(stringEndDate)) {
33      dateRange = new DateRange(DateUtils.stringToUtilsDate(
34        stringEndDate), -daysRange);
35      isCorrectRange = false;
36    } else if (DateUtils.checkDate(stringStartDate)) {
37      dateRange = new DateRange(DateUtils.stringToUtilsDate(
38        stringStartDate), daysRange);
39      isCorrectRange = false;

```

```

33     } else {
34         dateRange = new DateRange(-daysRange);
35     }
36 } catch (ParseException e) {
37     logger.error(" ERROR in getFilteredDocuments: " + e);
38     dateRange = new DateRange(-daysRange);
39 }
40
41 if (!isCorrectRange) {
42     model.addAttribute("responseMessage", ResponseEnum.
43         LIST_WARNING_1.getMessageKey());
44     model.addAttribute("isAWarningMessage", true);
45 }
46
47 return dateRange;
48 }

```

Listado B.8: Función getCreateClientUserView del controlador

```

1 @GetMapping("/user/create")
2 public String getCreateClientUserView() {
3     logger.debug("Entering GET: /user/create");
4
5     return "createClientUserView";
6 }

```

Listado B.9: Función createClientUser del controlador

```

1 @PostMapping("/user/create")
2 public String createClientUser(Model model,
3     @ModelAttribute ClientUser
4         clientUser,
5     @RequestParam("psw-repeat") String
6         passwordRepeat
7     ) throws Exception {
8
9     logger.debug(" - Entering clientUserService.createClientUser()
10         , from POST: /user/create");
11
12     ResponseEnum responseMessage = clientUserService.
13         createClientUser(clientUser, passwordRepeat);
14
15     model.addAttribute("responseMessage", responseMessage.
16         getMessageKey());
17     addColorToResponse(model, responseMessage);
18
19     return "createClientUserView";
20 }

```

Listado B.10: Función saveClientUser del controlador

```

1 @Transactional
2 public ClientUser saveClientUser(ClientUser clientUser) throws
   Exception {
3     try {
4         if (clientUser.getPassword() != null) {
5             logger.debug(" -- Entering encryptMessage()");
6             clientUser.setPassword(encryptFacade(clientUser.
               getPassword()));
7
8             logger.debug(" -- Entering randomAlphanumeric()");
9             clientUser.setConfirmationCode(RandomStringUtils.
               randomAlphanumeric(30));
10
11             clientUser.setStatus(0);
12             clientUser.setCreationDate(new Date());
13             clientUserRepository.save(clientUser);
14         } else {
15             throw new IllegalArgumentException("Password cannot
               be null");
16         }
17     } catch (Exception e) {
18         logger.error(" ERROR: Possible error in encryption,
               confirmation code generation "
19             + " or saveClientUser:" + e);
20         throw e;
21     }
22
23     return clientUser;
24 }

```

Listado B.11: Función getConfirmation del controlador

```

1 @GetMapping("/user/confirm")
2 public String getConfirmation(Model model,
3                               @RequestParam("email") String email,
4                               @RequestParam("confirmationCode")
                                   String confirmationCode) {
5
6     logger.debug(" - Entering clientUserService.confirmUser(),
               from GET: /user/confirm");
7     ResponseEnum responseMessage = clientUserService.confirmUser(
               email, confirmationCode);
8     logger.debug(" - Exiting clientUserService.confirmUser()");
9
10    model.addAttribute("responseMessage", responseMessage.
               getMessageKey());
11    addColorToResponse(model, responseMessage);
12

```

```
13     return "confirmationUser";
14 }
```

Listado B.12: Función confirmUser del servicio

```
1 @Transactional
2 public ResponseEnum confirmUser(String email,
3                                 String confirmationCode
4                                 ) {
5     ResponseEnum responseMessage;
6     ClientUser clientUser = clientUserRepository.
7         findByEmailAndConfirmationCodeAndStatus(email,
8         confirmationCode, 0);
9
10    if (clientUser != null) {
11        responseMessage = ResponseEnum.CONFIRM_SUCCESS;
12        clientUser.setStatus(1);
13        clientUser.setConfirmationDate(new Date());
14        clientUserRepository.save(clientUser);
15    } else {
16        logger.debug(" User not found, not correct or already
17            confirmed. ");
18        responseMessage = ResponseEnum.CONFIRM_WARNING;
19    }
20
21    return responseMessage;
22 }
```

Listado B.13: Función getLoginView del controlador

```
1 @GetMapping("/user/login")
2 public String getLoginView(Model model,
3                             HttpServletRequest request
4                             ) throws Exception {
5     String serverUrl = SharedProperties.get("serverUrl");
6     String requestUpdatePasswordUrl = serverUrl + "user/
7         requestUpdatePassword";
8     String createClientUserUrl = serverUrl + "user/create";
9     String errorMessage = (String) request.getSession().
10         getAttribute("error");
11
12     model.addAttribute("requestUpdatePasswordUrl",
13         requestUpdatePasswordUrl);
14     model.addAttribute("createClientUserUrl", createClientUserUrl);
15
16     if (errorMessage != null) {
17         ResponseEnum responseMessage = ResponseEnum.
18             LOGIN_WARNING_1;
19     }
20 }
```

```

15     model.addAttribute("responseMessage", responseMessage.
16         getMessageKey());
17     addColorToResponse(model, responseMessage);
18
19     // Elimina el mensaje de error de la sesión
20     request.getSession().removeAttribute("error");
21 }
22
23     return "loginView";
24 }

```

Listado B.14: Función getRequestUpdatePasswordView del controlador

```

1 @GetMapping("/user/requestUpdatePassword")
2 public String getRequestUpdatePasswordView() {
3     return "requestUpdatePasswordView";
4 }

```

Listado B.15: Función requestUpdatePassword del controlador

```

1 @PostMapping("user/requestUpdatePassword")
2 public String requestUpdatePassword(Model model,
3     @RequestParam("email") String email
4     ) throws Exception {
5
6     ResponseEnum responseMessage = clientUserService.
7         sendUpdatePasswordUrl(email);
8
9     model.addAttribute("responseMessage", responseMessage.
10         getMessageKey());
11     addColorToResponse(model, responseMessage);
12
13     return "requestUpdatePasswordView";
14 }

```

Listado B.16: Función sendUpdatePasswordUrl del servicio

```

1 public ResponseEnum sendUpdatePasswordUrl(String email) throws
2     Exception {
3     ResponseEnum responseMessage;
4     ClientUser clientUser = clientUserRepository.findByEmail(email);
5
6     if (clientUser != null && clientUser.getStatus() == 1) {
7         String path = "user/updatePassword?email=";
8         String urlUpdatePassword = this.getConfirmationUrl(
9             clientUser, path);
10
11         logger.debug(" - Entering sendMail");
12     }
13 }

```

```

10     String subject = " Change the password of your SIDManager
11         user. ";
12     String body = " Click on the next URL if you want to
13         change your password: ";
14     Mail.sendMail(clientUser.getEmail(), urlUpdatePassword,
15         subject, body);
16     logger.debug(" - Exiting sendMail");
17
18     responseMessage = ResponseEnum.UPDATE_PSW_SUCCESS_1;
19 } else {
20     responseMessage = ResponseEnum.UPDATE_PSW_WARNING_1;
21     logger.info(" The user does not exist or is not confirmed.
22         ");
23 }
24
25 return responseMessage;
26 }

```

Listado B.17: Función getConfirmationUrl del servicio

```

1 private String getConfirmationUrl(ClientUser clientUser,
2     String path
3 ) throws Exception {
4     StringBuilder confirmationUrl = new StringBuilder();
5     String serverUrl = SharedProperties.get("serverUrl");
6
7     confirmationUrl.append(serverUrl);
8     confirmationUrl.append(path);
9     confirmationUrl.append(clientUser.getEmail());
10    confirmationUrl.append("&confirmationCode=");
11    confirmationUrl.append(clientUser.getConfirmationCode());
12
13    return confirmationUrl.toString();
14 }

```

Listado B.18: Función getUpdatePasswordView del controlador

```

1 @GetMapping("/user/updatePassword")
2 public String getUpdatePasswordView(Model model,
3     HttpSession session,
4     @RequestParam("email") String email,
5     @RequestParam("confirmationCode") String confirmationCode
6 ) throws Exception {
7     boolean shouldFormBeShown = false;
8     ResponseEnum responseMessage = clientUserService.
9         getUpdatePasswordView(email, confirmationCode);
10
11     if (responseMessage.getStatus() == ResponseEnum.
12         UPDATE_PSW_SUCCESS_2.getStatus()) {

```

```

11     shouldFormBeShown = true;
12 } else {
13     model.addAttribute("responseMessage", responseMessage.
14         getMessageKey());
15     addColorToResponse(model, responseMessage);
16 }
17
18 model.addAttribute("shouldFormBeShown", shouldFormBeShown);
19
20 //Save the email to send it later in the post
21 session.setAttribute("email", email);
22
23 return "updatePasswordView";
24 }

```

Listado B.19: Función getUpdatePasswordView del servicio

```

1 public ResponseEnum getUpdatePasswordView(String email,
2                                           String confirmationCode
3                                           ) {
4     ResponseEnum responseMessage;
5     ClientUser clientUser
6         = clientUserRepository.
7         findByEmailAndConfirmationCodeAndStatus(
8             email, confirmationCode, 1);
9
10    if (clientUser != null) {
11        responseMessage = ResponseEnum.UPDATE_PSW_SUCCESS_2;
12    } else {
13        responseMessage = ResponseEnum.UPDATE_PSW_WARNING_2;
14        logger.info(responseMessage.getMessageKey());
15    }
16
17    return responseMessage;
18 }

```

Listado B.20: Función updatePassword del controlador

```

1 @PostMapping("/user/updatePassword")
2 public String updatePassword( Model model,
3                               HttpSession session,
4                               @ModelAttribute ClientUser clientUser,
5                               @RequestParam("psw-repeat") String passwordRepeat
6                               ) throws Exception {
7     String loginUrl = SharedProperties.get("serverUrl") + "
8         user/login";
9     String email = (String) session.getAttribute("email");
10    boolean shouldFormBeShown = true;
11    ResponseEnum responseMessage = clientUserService.
12        updatePassword(email, clientUser, passwordRepeat);

```



```
11
12     if (responseMessage.getStatus() == ResponseEnum.
13         UPDATE_PSW_SUCCESS_3.getStatus()) {
14         shouldFormBeShown = false;
15         model.addAttribute("loginUrl", loginUrl);
16     }
17
18     model.addAttribute("responseMessage", responseMessage.
19         getMessageKey());
20     model.addAttribute("shouldFormBeShown", shouldFormBeShown)
21     ;
22     addColorToResponse(model, responseMessage);
23
24     return "updatePasswordView";
25 }
```

Listado B.21: Función updatePassword del servicio

```
1 public ResponseEnum updatePassword(String email,
2     ClientUser clientUser,
3     String passwordRepeat
4 ) throws Exception {
5     ResponseEnum responseMessage;
6     String password = clientUser.getPassword();
7
8     if (password != null && password.equals(
9         passwordRepeat)) {
10         // Check if the user exists in the
11         // database
12         ClientUser existingUser = this.
13             getClientUser(email);
14
15         if (existingUser != null) {
16             existingUser.setPassword(
17                 encryptFacade(password));
18             clientUserRepository.save(
19                 existingUser);
20
21             responseMessage = ResponseEnum.
22                 UPDATE_PSW_SUCCESS_3;
23         } else {
24             responseMessage = ResponseEnum.
25                 UPDATE_PSW_WARNING_4;
26         }
27     } else {
28         responseMessage = ResponseEnum.
29             UPDATE_PSW_WARNING_3;
30     }
31 }
```

```
24         return responseMessage;
25     }
```

Listado B.22: Clase DocumentModel

```
1  package com.isban.scf.sid.model;
2
3  import javax.persistence.Column;
4  import javax.persistence.Entity;
5  import javax.persistence.Id;
6  import javax.persistence.Table;
7
8  @Entity
9  @Table(name = "documents") // Nombre de la tabla en la base de
    datos
10 public class Document {
11     @Id
12     @Column(name = "CLIENT_DOC_ID")
13     private Long clientDocId;
14
15     @Column(name = "CLIENT_APP_ID")
16     private Long clientAppId;
17
18     @Column(name = "PROVIDER_ID")
19     private String providerId;
20
21     @Column(name = "SENT_AT")
22     private String sentAt;
23
24     @Column(name = "RECEIVED_AT")
25     private String receivedAt;
26
27     // Getters y setters
28     public Long getClientDocId() {
29         return clientDocId;
30     }
31
32     public void setClientDocId(Long clientDocId) {
33         this.clientDocId = clientDocId;
34     }
35
36     public Long getClientAppId() {
37         return clientAppId;
38     }
39
40     public void setClientAppId(Long clientAppId) {
41         this.clientAppId = clientAppId;
42     }
43 }
```

```
44 public String getProviderId() {
45     return providerId;
46 }
47
48 public void setProviderId(String providerId) {
49     this.providerId = providerId;
50 }
51
52 public String getSentAt() {
53     return sentAt;
54 }
55
56 public void setSentAt(String sentAt) {
57     this.sentAt = sentAt;
58 }
59
60 public String getReceivedAt() {
61     return receivedAt;
62 }
63
64 public void setReceivedAt(String receivedAt) {
65     this.receivedAt = receivedAt;
66 }
```

Listado B.23: Clase SecurityConfig

```
1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     private final UserDetailsService userDetailsService;
6
7     public SecurityConfig(UserDetailsService userDetailsService) {
8         this.userDetailsService = userDetailsService;
9     }
10
11     @Bean
12     public SecurityFilterChain securityFilterChain(
13         HttpSecurity http) throws Exception {
14         http
15             .authorizeHttpRequests(authorizeRequests ->
16                 authorizeRequests
17                     .antMatchers("/user/login",
18                         "/user/create",
19                         "/user/confirm",
20                         "/user/requestUpdatePassword",
21                         "/user/updatePassword",
22                         "/user/getAll",
23                         "/user/deleteAll",
```

```

23         "/home",
24         "/css/**",
25         "/img/**",
26         "/js/**"
27     ).permitAll() // Permitir acceso sin
                autenticación
28     .anyRequest().authenticated() // Cualquier
                otra ruta requiere autenticación
29 )
30 .formLogin(formLogin ->
31     formLogin
32     .loginPage("/user/login") // Página de
                login personalizada
33     .defaultSuccessUrl("/list", true) //
                Redirigir al home después del login
                exitoso
34     .permitAll()
35     .failureHandler(
                authenticationFailureHandler())
36 )
37 .logout(logout ->
38     logout
39     .logoutUrl("/logout")
40     .logoutSuccessUrl("/user/login?logout") //
                Redirigir a login después del logout
41     .invalidateHttpSession(true)
42     .deleteCookies("JSESSIONID")
43     .permitAll()
44 )
45 .exceptionHandling(exceptionHandling ->
46     exceptionHandling.accessDeniedPage("/user/
                accessDenied")); // Manejo de accesos
                denegados
47
48     return http.build();
49 }
50
51 protected void configure(AuthenticationManagerBuilder auth)
52     throws Exception {
53     auth.userDetailsService(userDetailsService).
        passwordEncoder(passwordEncoder());
54 }
55
56 @Bean
57 public PasswordEncoder passwordEncoder() {
58     return new BCryptPasswordEncoder();
59 }
60
61 @Bean

```

```
61     public AuthenticationFailureHandler
        authenticationFailureHandler() {
62         return new CustomAuthenticationFailureHandler();
63     }
64
65 }
```

Listado B.24: Clase CustomUserDetailsService

```
1
2 @Service
3 public class CustomUserDetailsService implements
    UserDetailsService {
4
5     private final ClientUserRepository clientUserRepository;
6
7     public CustomUserDetailsService(ClientUserRepository
        userRepository) {
8         this.clientUserRepository = userRepository;
9     }
10
11     @Override
12     public UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException {
13         ClientUser user = clientUserRepository.findByEmail(
            username);
14
15         if (user == null) {
16             throw new UsernameNotFoundException("User not found");
17         }
18
19         CustomUserDetails customUserDetails = new
            CustomUserDetails(user);
20
21         return customUserDetails;
22     }
23 }
```

Listado B.25: Clase CustomUserDetailsService modificado

```
1
2 @Service
3 public class CustomUserDetailsService implements
    UserDetailsService {
4
5     private final ClientUserRepository clientUserRepository;
6
7     public CustomUserDetailsService(ClientUserRepository
        userRepository) {
```

```

8         this.clientUserRepository = userRepository;
9     }
10
11     @Override
12     public UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException {
13         ClientUser user = clientUserRepository.findByEmail(
            username);
14
15         if (user == null) {
16             throw new UsernameNotFoundException("User not found");
17         }
18
19         CustomUserDetails customUserDetails = new
            CustomUserDetails(user);
20
21         return customUserDetails;
22     }
23 }

```

Listado B.26: Clase CustomAuthenticationFailureHandler

```

1 @Component
2 public class CustomAuthenticationFailureHandler implements
    AuthenticationFailureHandler {
3
4     @Override
5     public void onAuthenticationFailure(HttpServletRequest request
        , HttpServletResponse response, AuthenticationException
        exception)
6         throws IOException, ServletException {
7         // Establece un mensaje de error en la sesión
8         request.getSession().setAttribute("error", "Login error");
9
10        // Redirige de nuevo a la página de login
11        response.sendRedirect("/user/login");
12    }
13
14 }

```

Listado B.27: Función encryptMessage de Encryptor

```

1 public static String encryptMessage(String message) throws
    Exception {
2     if (message == null) {
3         throw new IllegalArgumentException("Message to be
            encrypted cannot be null");
4     }
5     byte[] key = SharedProperties.getEncryptKey().getBytes();

```

```

6
7 Cipher cipher = Cipher.getInstance(ALGORITHM);
8 SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
9 cipher.init(Cipher.ENCRYPT_MODE, secretKey);
10 byte[] encryptedMessage = cipher.doFinal(message.getBytes());
11
12 return Base64.getEncoder().encodeToString(encryptedMessage);
13 }

```

Listado B.28: Función decryptMessage de Encryptor

```

1 public static String decryptMessage(String encryptedMessage)
2     throws Exception {
3     byte[] key = SharedProperties.getEncryptKey().getBytes();
4
5     byte[] messageBytes = Base64.getDecoder().decode(
6         encryptedMessage);
7     Cipher cipher = Cipher.getInstance(ALGORITHM);
8     SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
9     cipher.init(Cipher.DECRYPT_MODE, secretKey);
10    byte[] decryptedBytes = cipher.doFinal(messageBytes);
11
12    return new String(decryptedBytes);
13 }

```

Listado B.29: La función getDataSource de DataSourceConfig

```

1 @Bean
2 @Primary
3 public DataSource getDataSource() {
4     logger.debug(" --- Creating driver for database connection ---");
5
6     DriverManagerDataSource dataSource = new
7         DriverManagerDataSource();
8     dataSource.setUrl(dbUrl);
9     dataSource.setUsername(dbUsername);
10
11     try {
12         String decryptedPassword = Encryptor.decryptMessage(
13             encryptedPassword);
14         dataSource.setPassword(decryptedPassword);
15     } catch (Exception e) {
16         logger.error(" ERROR while decrypting the database
17             password (check application.properties): ", e);
18         throw new RuntimeException();
19     }
20
21     logger.debug(" --- Driver was successfully created --- ");
22 }

```

```

19     return dataSource;
20 }

```

Listado B.30: El contenido html para mostrar la tabla de documentos

```

1 <table id="example" class="table table-striped">
2   <thead>
3     <tr>
4       <th colspan="5" id="tableHead"> Documents in the
        database </th>
5     </tr>
6     <tr>
7       <th id="columnHeader1"> Client app id </th>
8       <th id="columnHeader2"> Client doc id </th>
9       <th id="columnHeader3"> Provider id </th>
10      <th id="columnHeader4"> Sent at </th>
11      <th id="columnHeader5"> Received at </th>
12    </tr>
13  </thead>
14  <tbody>
15    <!-- Este bloque se repetirá para cada documento en la
        lista -->
16    <tr th:each="document : ${documents}">
17      <td th:text="${document.clientAppId}"></td>
18      <td th:text="${document.clientDocId}"></td>
19      <td th:text="${document.providerId}"></td>
20      <td th:text="${document.sentAt}"></td>
21      <td th:text="${document.receivedAt ?: 'Not received
        '}"></td>
22    </tr>
23  </tbody>
24 </table>

```

Listado B.31: El contenido html para conectar los links de Bootstrap 5

```

1 <!-- CSS -->
2 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/
    libs/twitter-bootstrap/5.3.0/css/bootstrap.min.css">
3 <link rel="stylesheet" href="https://cdn.datatables.net/1.13.4/css
    /dataTables.bootstrap5.min.css">
4
5 <!-- JavaScript -->
6 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script
    >
7 <script src="https://cdnjs.cloudflare.com/ajax/libs/twitter-
    bootstrap/5.3.0/js/bootstrap.bundle.min.js"></script>
8 <script src="https://cdn.datatables.net/1.13.4/js/jquery.
    dataTable.min.js"></script>
9 <script src="https://cdn.datatables.net/1.13.4/js/dataTables.
    bootstrap5.min.js"></script>

```


Listado B.32: El contenido html para enviar un mensaje al front-end con Thymeleaf

```

1 <!-- Show the message if it exists / paint it red if it is a
   warning / print the one from messages.properties with that name
   -->
2 <div id="errorMessageModal" class="modal" th:if="${responseMessage
   }">
3     <div class="modal-content"
4         th:style="${isAWarningMessage ?
5             'background-color:red;' :
6             'background-color:green;'}">
7         <span class="close-button">&times;</span>
8         <p th:text="#${${responseMessage}}"></p>
9     </div>
10 </div>

```

Listado B.33: La clase Mail

```

1 public class Mail {
2     private static final Logger logger = LogManager.getLogger(
3         Mail.class);
4
5     /*
6     public void main(String [] args) throws Exception {
7         logger.debug(" - Entering sendMail");
8         String subject = "Confirm your sign up in SIDManager";
9         String body = "Click on the next URL to activate your
10             SIDManager user: ";
11         Mail.sendMail("daleaenter00@gmail.com", "urlConfirmation",
12             subject, body);
13     }
14     */
15
16     /**
17     * This method creates the session, it also specify the
18     * server properties
19     * and set the message and its content (that includes the
20     * receivers,
21     * the body of the mail...).
22     * Finally the method send the mail.
23     * @param confirmationCode
24     * @param receiver
25     *
26     * @throws Exception If a method exception occurred
27     */
28     public static void sendMail(String receiver,
29         String urlConfirmation,

```

```

26         String subject,
27         String body) throws Exception {
28     logger.debug("Entering Mail");
29
30     final String sender = SharedProperties.get( "mailSender" )
31         ;
32
33     final String password = SharedProperties.get( "
34         mailPassword" );
35     // Crear sesión de correo
36     Session session = getSession(sender, password);
37
38     // Get session logging in with server properties
39     //Session session = Session.getInstance(
40         getServerProperties());
41
42     try {
43         // Create email message
44         Message message = createMessage(session, sender,
45             receiver, urlConfirmation, subject, body);
46
47         // Send email
48         Transport.send(message);
49
50         logger.debug("The mail was sent successfully.");
51
52     } catch (MessagingException e) {
53         logger.error("There was an error sending the email
54             : " + e.getMessage());
55         logger.error("Status of the VARIABLES: "
56             + " | session: " + session
57             + " | sender: " + sender
58             + " | receiver: " + receiver
59             + " | urlConfirmation: " +
60                 urlConfirmation
61             + " | subject: " + subject
62             + " | body: " + body
63         );
64         throw new Exception();
65     }
66
67     logger.debug("Exiting Mail");
68 }
69
70 /**
71  * This method define who is the sender and receiver,
72  * it also specify the mail subject, body and attachment.
73  *
74  * @param session A session for the logged sender in

```

```
        Session format.
69      * @param sender A sender which is going to send the email
        in string format.
70      * @param receiver A receiver of the email in String
        format.
71      * @param confirmationCode A code for confirm a user in
        String format.
72      *
73      * @throws AddressException If an error with the address
        occurs
74      * @throws MessagingException If an error with the message
        occurs
75      *
76      * @return the email message in Message format
77      */
78      private static Message createMessage
79      (Session session, String sender, String receiver, String
        urlConfirmation, String subject, String body)
80      throws AddressException, MessagingException {
81
82      logger.debug("Entering setMessageAndItsContent");
83
84      Message message = new MimeMessage(session);
85
86      // Define who the sender is
87      message.setFrom(new InternetAddress(sender));
88
89      message.setRecipient(Message.RecipientType.TO, new
        InternetAddress(receiver));
90
91      // Set email subject
92      //message.setSubject("Confirm your sign up in SIDManager")
93      ;
94      message.setSubject(subject);
95
96      // Email body
97      //String content = "Click on the next URL to activate your
        SIDManager user: " + urlConfirmation;
98      String content = body + urlConfirmation;
99
100     MimeBodyPart mimeBodyPart = new MimeBodyPart();
101     mimeBodyPart.setContent(content, "text/html; charset=utf-8
        ");
102
103         // Create multipart email, mix the attach to the
        email body.
104     MimeMultipart multipart = new MimeMultipart();
105     multipart.addBodyPart(mimeBodyPart);
```

```

106         message.setContent(multipart);
107
108         logger.debug("Exiting setMessageAndItsContent");
109
110         return message;
111     }
112
113
114
115     /**
116     * This method get the session of the sender with
117         authentication.
118     *
119     * @param sender A sender which is going to send the email
120         in string format.
121     * @param password The password of the sender email.
122     *
123     * @throws Exception If an error with the session occurs
124     *
125     * @return the session in Session format
126     */
127     private static Session getSession (final String sender,
128         final String password) throws Exception {
129
130         // Set mail server settings
131         Properties props = getServerProperties();
132
133         // Get session logging in with user and password
134         Session session = Session.getInstance(props, new
135             Authenticator() {
136                 protected PasswordAuthentication
137                     getPasswordAuthentication() {
138                     return new PasswordAuthentication(sender,
139                         password);
140                 }
141             });
142         return session;
143     }
144
145     /**
146     * This method get the properties needed for the session (
147         port, host...).
148     *
149     * @return the properties needed for the session (port and
150         host).
151     *
152     * @exception Exception If an error with the properties
153         occurs

```

```
146      */
147      private static Properties getServerProperties() throws
          Exception {
148          // Get the port and host
149          String host = SharedProperties.get("mailHost");
150          String port = SharedProperties.get("mailPort");
151
152          // Server properties are specified here
153          Properties props = new Properties();
154          props.put("mail.smtp.host", host);
155          props.put("mail.smtp.port", port);
156          props.put("mail.smtp.auth", "true"); //Si se requiere
              autentificacion y password
157          props.put("mail.smtp.starttls.enable", "true");
158
159          //props.put("mail.smtp.socketFactory.class", "javax.net.
              ssl.SSLSocketFactory");
160          //props.put("mail.smtp.socketFactory.fallback", "false");
161
162          return props;
163      }
164  }
```

Listado B.34: La clase ResponseEnum

```
1  public enum ResponseEnum {
2      SIGN_UP_SUCCESS("singup.success", 210),
3      CONFIRM_SUCCESS("confirm.success", 220),
4      LOGIN_ERROR_1("login.error.1", 430);
5
6      private String messageKey;
7      private int status;
8
9      private ResponseEnum(String messageKey, int status) {
10         this.messageKey = messageKey;
11         this.status = status;
12     }
13
14     public String getMessageKey() {
15         return messageKey;
16     }
17
18     public int getStatus() {
19         return status;
20     }
21 }
```

Listado B.35: La clase LenguajeConfig

```
1 @Configuration
2 public class LenguajeConfig implements WebMvcConfigurer {
3
4     @Bean
5     public SessionLocaleResolver localeResolver() {
6         SessionLocaleResolver slr = new SessionLocaleResolver();
7         slr.setDefaultLocale(Locale.US); // Idioma por defecto:
8             inglés
9         return slr;
10    }
11
12    @Bean
13    public LocaleChangeInterceptor localeChangeInterceptor() {
14        LocaleChangeInterceptor lci = new LocaleChangeInterceptor
15            ();
16        lci.setParamName("lang"); // Parámetro para cambiar el
17            idioma
18        return lci;
19    }
20
21    @Override
22    public void addInterceptors(InterceptorRegistry registry) {
23        registry.addInterceptor(localeChangeInterceptor());
24    }
25 }
```

Lista de acrónimos

API Application Programming Interface. 14, 17

CAPTCHA Completely Automated Public Turing test to tell Computers and Humans Apart. 21

CBC Cipher Block Chaining. 20

CPU Central Processing Unit. 21

CRUD Create, Read, Update, Delete. 18

CSRF Cross-Site Request Forgery. 21

CSS Cascading Style Sheets. 13

ECB Electronic Codebook. 20

ECC Elliptic Curve Cryptography. 20

GCM Galois/Counter Mode. 20

GET Hypertext Transfer Protocol GET Method. 14

HTML HyperText Markup Language. 13

Java EE Java Enterprise Edition. 18

JPA Java Persistence API. 18

JSX JavaScript XML. 16

MVC Model-View-Controller. 16

POST Hypertext Transfer Protocol POST Method. 14

PUT Hypertext Transfer Protocol PUT Method. 14

REST Representational State Transfer. 14

RSA Rivest–Shamir–Adleman (Cryptosystem). 20

SPA Single Page Application. 14

SQL Structured Query Language. 13

UML Unified Modeling Language. 22

URL Uniform Resource Locator. 14

VDOM Virtual Document Object Model. 16

XML eXtensible Markup Language. 19